



Projeto2: Clínica Veterinária

Descrição	Criar um projeto com django, django rest + ionic
Tags	Angular Django Django Rest Ionic Python

Clinica Veterinaria é um projeto simples desenvolvido com django para agendar consultas medicas.

▼ Django

Funcionalidades Básicas:

1. **Cadastro de Animais:** Nome, espécie, raça, idade, peso, dono, etc.
2. **Cadastro de Clientes (Donos):** Nome, telefone, endereço, e-mail, etc.
3. **Registro de Consultas:** Data, motivo, veterinário, observações, animal.
4. **Cadastro de Veterinários:** Nome, CRMV, especialidade, contato.
5. **Agenda de Consultas:** Calendário para gerenciar consultas.

▼ Parte 1

Estrutura do Projeto

1. Criação do Projeto e App

```
django-admin startproject ClinicaDoBicho
cd ClinicaDoBicho
python manage.py startapp core
```

2. Configuração do settings.py

- Adicione `core` ao `INSTALLED_APPS`

3. Modelos (`models.py` em `core`)

```
from django.db import models

# Tabela de clientes
class Cliente(models.Model):
    nome = models.CharField(max_length=100)
    telefone = models.CharField(max_length=15)
    email = models.EmailField()
    endereco = models.TextField()

    def __str__(self):
        return self.nome

# Tabela de Animais
class Animal(models.Model):
    ESPECIES = [
        ('C', 'Cachorro'),
        ('G', 'Gato'),
        ('O', 'Outros'),
    ]
    nome = models.CharField(max_length=100)
    especie = models.CharField(max_length=1, choices=ESPECIES)
    raca = models.CharField(max_length=100)
    idade = models.PositiveIntegerField(null=True, blank=True)
    peso = models.DecimalField(max_digits=5, decimal_places=2, null=True, blank=True)
```

```

    dono = models.ForeignKey(Cliente, on_delete=models.CASCADE)

    def __str__(self):
        return f"{self.nome} ({self.especie})"

# Cadastro de Doutores
class MedicoVeterinario(models.Model):
    nome = models.CharField(max_length=100)
    crmv = models.CharField(max_length=20)
    especialidade = models.CharField(max_length=100)
    contato = models.CharField(max_length=15)

    def __str__(self):
        return self.nome

# Agenda consulta
class Consulta(models.Model):
    animal = models.ForeignKey(Animal, on_delete=models.CASCADE)
    veterinario = models.ForeignKey(MedicoVeterinario, on_delete=models.
SET_NULL, null=True)
    data = models.DateTimeField()
    motivo = models.TextField()
    observacoes = models.TextField(blank=True)

    def __str__(self):
        return f"Consulta de {self.animal.nome} com {self.veterinario.nome}
em {self.data}"

```

4. Criação das Migrações

```

python manage.py makemigrations
python manage.py migrate

```

5. Admin (`admin.py`)

```

from django.contrib import admin
from .models import Cliente, Animal, Veterinario, Consulta

admin.site.register(Cliente)
admin.site.register(Animal)
admin.site.register(Veterinario)
admin.site.register(Consulta)

```

6. URLs (`urls.py`)

```

from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('core.urls')), # Criar o arquivo `urls.py` no app
]

```

7. Views e Templates (CRUD)

- Criar Views (CRUD básico para clientes, animais, veterinários e consultas).
- Usar **CBVs (Class-Based Views)** ou **FBVs (Function-Based Views)**.
- Criar **templates** com Bootstrap para formulários e listas.

Exemplo de View:

```

from django.shortcuts import render, get_object_or_404
from .models import Animal

def lista_animais(request):
    animais = Animal.objects.all()
    return render(request, 'lista_animais.html', {'animais': animais})

```

Exemplo de Template (`templates/lista_animais.html`):

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <h1>Lista de Animais</h1>
  <table>
    <tr>
      <th>Nome</th>
      <th>Espécie</th>
      <th>Raça</th>
    </tr>
    {% for animal in animais %}
    <tr>
      <td>{{ animal.nome }}</td>
      <td>{{ animal.get_especie_display }}</td>
      <td>{{ animal.raca }}</td>
    </tr>
    {% endfor %}
  </table>
</body>
</html>

```

8. URLs do App (`core/urls.py`)

```

from django.urls import path
from . import views

urlpatterns = [

```

```
path('animais/', views.lista_animais, name='lista_animais'),  
]
```

Você pode ver que vai dar um erro: **Não encontrou o template HTML.**

Para tratar isso precisamos dizer ao projeto onde ficam as pastas `templates` e `statics`.

Settings - Statics

```
# Statics  
STATIC_ROOT = os.path.join(BASE_DIR,'static')  
STATIC_URL = '/static/'  
  
MEDIA_ROOT=os.path.join(BASE_DIR,'media')  
MEDIA_URL = '/media/'  
  
# Language  
LANGUAGE_CODE = 'pt-br'  
TIME_ZONE = 'America/Sao_Paulo'  
USE_I18N = True  
USE_L10N = True  
USE_TZ = True  
  
# Dir  
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))  
TEMPLATE_DIR = os.path.join(BASE_DIR,'templates')  
  
# Database  
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': os.path.join(BASE_DIR, 'db.sqlite3'),  
    }  
}
```

9. Criar Template Base

Criar template base `base.html`:

```
{% load static %}
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1,
shrink-to-fit=no">

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootst
trap.min.css" rel="stylesheet">

  <title>My Site</title>
</head>

<body>
  <div class="container my-5">
    {% block content %}
    {% endblock %}
  </div>

  <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootst
rap.min.js"></script>
</body>
</html>
```

10. Validação no Modelo Consulta

```
from django.core.exceptions import ValidationError

class Consulta(models.Model):
```

```

    animal = models.ForeignKey(Animal, on_delete=models.CASCADE)
    veterinario = models.ForeignKey(Veterinario, on_delete=models.SET_NULL, null=True)
    data = models.DateTimeField()
    motivo = models.TextField()
    observacoes = models.TextField(blank=True)

    def clean(self):
        if Consulta.objects.filter(data=self.data, veterinario=self.veterinario).exists():
            raise ValidationError('Já existe uma consulta agendada para este horário com este veterinário.')

    def save(self, *args, **kwargs):
        self.clean()
        super().save(*args, **kwargs)

    def __str__(self):
        return f"Consulta de {self.animal.nome} com {self.veterinario.nome} em {self.data}"

```

Mensagens de Erro no Formulário

```

from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import ConsultaForm

def agendar_consulta(request):
    if request.method == 'POST':
        form = ConsultaForm(request.POST)
        if form.is_valid():
            try:
                form.save()
                messages.success(request, 'Consulta agendada com sucesso!')
                return redirect('lista_consultas')
            except ValidationError as e:

```

```

        messages.error(request, e.message)
    else:
        form = ConsultaForm()
    return render(request, 'agendar_consulta.html', {'form': form})

```

Formulário com Validação

```

from django import forms
from .models import Consulta

class ConsultaForm(forms.ModelForm):
    class Meta:
        model = Consulta
        fields = ['animal', 'veterinario', 'data', 'motivo', 'observacoes']

    def clean(self):
        cleaned_data = super().clean()
        data = cleaned_data.get('data')
        veterinario = cleaned_data.get('veterinario')

        if Consulta.objects.filter(data=data, veterinario=veterinario).exists():
            raise forms.ValidationError('Já existe uma consulta agendada neste
            horário para este veterinário.')
        return cleaned_data

```

base.html (mensagens)

```

{% if messages %}
    <div>
        {% for message in messages %}
            <div class="alert alert-{{ message.tags }}" alert-dismissible fade show
            w" role="alert">
                {{ message }}
                <button type="button" class="btn-close" data-bs-dismiss="alert"
                aria-label="Close"></button>
            </div>
        {% endfor %}
    </div>

```

```
{% endfor %}
</div>
{% endif %}
```

settings.py (Messages Bootstrap)

```
from django.contrib.messages import constants

MESSAGE_TAGS = {
    constants.ERROR: 'alert-danger',
    constants.WARNING: 'alert-warning',
    constants.DEBUG: 'alert-danger',
    constants.SUCCESS: 'alert-success',
    constants.INFO: 'alert-info',
}
```

Template Agendamento (`agendar_consulta.html`)

```
<!-- Formulário de agendamento →
<form method="POST">
    {% csrf_token %}
    {{form}}
    <button type="submit" class="btn btn-primary">Agendar</button>
    <a href="{% url 'lista_consultas' %}" class="btn btn-secondary">Cancel
    elar</a>
</form>
```

Melhoria

```
widgets = {
    'data': forms.DateTimeInput(attrs={'type': 'datetime-local'}),
}

def clean(self):
    # A validação fica aqui, no formulário.
    cleaned_data = super().clean()
```

```

data = cleaned_data.get('data')
veterinario = cleaned_data.get('veterinario')

# Garante que ambos os campos existem antes de tentar a consulta
if data and veterinario:
    # Exclui a própria instância se estivermos editando uma consulta e
    # existente
    # Isso evita que o formulário dê erro ao salvar uma edição sem alte
    # rar o horário.
    qs = Consulta.objects.filter(data=data, veterinario=veterinario)
    if self.instance.pk:
        qs = qs.exclude(pk=self.instance.pk)

    if qs.exists():
        # Esta é a forma padrão de adicionar um erro a um campo espec
        # ífico ou ao formulário geral.
        raise forms.ValidationError(
            'Já existe uma consulta agendada neste horário para este vete
            rinário.'
        )

    return cleaned_data

```

11. Lista de consultas

views.py

```

from django.shortcuts import render
from .models import Consulta

def lista_consultas(request):
    consultas = Consulta.objects.all().order_by('data')
    return render(request, 'lista_consultas.html', {'consultas': consultas})

```

urls.py

```

from django.urls import path
from . import views

urlpatterns = [
    path('consultas/', views.lista_consultas, name='lista_consultas'),
]

```

html

```

<h1>Lista de Consultas</h1>
<table class="table table-striped">
  <thead>
    <tr>
      <th>Animal</th>
      <th>Veterinário</th>
      <th>Data</th>
      <th>Motivo</th>
      <th>Observações</th>
    </tr>
  </thead>
  <tbody>
    {% for consulta in consultas %}
      <tr>
        <td>{{ consulta.animal.nome }}</td>
        <td>{{ consulta.veterinario.nome }}</td>
        <td>{{ consulta.data|date:"d/m/Y H:i" }}</td>
        <td>{{ consulta.motivo }}</td>
        <td>{{ consulta.observacoes }}</td>
      </tr>
    {% empty %}
      <tr>
        <td colspan="5">Nenhuma consulta cadastrada.</td>
      </tr>
    {% endfor %}
  </tbody>
</table>

```

```
</tbody>
</table>
```

Pode da uma melhoria na lista de animais tambem.

lista_animais.html

```
{% extends 'base.html' %}
{% block content %}
<div class="container my-5">
  <h1 class="mb-4">Lista de Animais</h1>
  <table class="table table-striped table-bordered">
    <thead class="table-dark">
      <tr>
        <th>Nome</th>
        <th>Espécie</th>
        <th>Raça</th>
      </tr>
    </thead>
    <tbody>
      {% for animal in animais %}
      <tr>
        <td>{{ animal.nome }}</td>
        <td>{{ animal.get_especie_display }}</td>
        <td>{{ animal.raca }}</td>
      </tr>
      {% empty %}
      <tr>
        <td colspan="3" class="text-center">Nenhum animal cadastrad
o.</td>
      </tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
```

12. Vamos colocar um template inicial na rota /

urls.py

```
from django.urls import path
from django.views.generic import TemplateView

urlpatterns = [
    path('',
        TemplateView.as_view(template_name='inicio.html'), name='home'),
]
```

inicio.html

```
{% extends 'base.html' %}
{% block content %}
<div class="d-flex justify-content-center align-items-center" style="height: 80vh;">
    <div class="card text-center shadow p-4" style="width: 18rem;">
        <div class="card-body">
            <h5 class="card-title">Consultas</h5>
            <a href="{% url 'lista_consultas' %}" class="btn btn-primary">Ver
Consultas</a>
        </div>
    </div>
</div>
{% endblock %}
```

Formulário de agendar visita podemos adicionar classe bootstrap em todos os campos.

```
def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)
```

```
for field in self.fields.values():
    field.widget.attrs.update({'class': 'form-control'})
```

13. Melhorada no form de agendar

Buscar cliente antes de abrir consulta se cliente não existir a gente cria.

agendar_consulta.html

```
<!-- Buscar cliente →
<form class="row g-3 mb-4" method="post">
  {% csrf_token %}
  <div class="col-auto">
    <label for="cpfInput" class="visually-hidden">CPF do Cliente</label>
    <input type="text" class="form-control" id="cpfInput" name="cpf" placeholder="Digite o CPF">
  </div>
  <div class="col-auto">
    <button type="submit" name="buscar" class="btn btn-primary">Buscar Cliente</button>
  </div>
</form>
```

views.py

```
def agendar_consulta(request):
    cliente = None
    animais = None

    if request.method == 'POST':
        # botão buscar cliente
        if "buscar" in request.POST:
            cpf = request.POST.get("cpf")
            try:
                cliente = Cliente.objects.get(cpf=cpf)
                animais = cliente.animais.all()
```

```

        form = ConsultaForm()
    except Cliente.DoesNotExist:
        messages.error(request, "Cliente não encontrado.")
        form = ConsultaForm()

    # botão salvar consulta
    elif "salvar" in request.POST:
        form = ConsultaForm(request.POST)
        if form.is_valid():
            form.save()
            messages.success(request, 'Consulta agendada com sucesso!')
            return redirect('lista_consultas')
        else:
            messages.error(request, 'Erro ao agendar consulta.')
    else:
        form = ConsultaForm()

    return render(request, 'agendar_consulta.html', {
        'form': form,
        'cliente': cliente,
        'animais': animais
    })

```

```

{% if cliente %}
<!-- Dados do cliente -->
<div class="card shadow-sm mb-4">
    <div class="card-body">
        <h5 class="card-title">Cliente encontrado: {{ cliente.cpf }}</h5>
        <p><strong>Nome:</strong> {{ cliente.nome }}</p>
        <p><strong>Telefone:</strong> {{ cliente.telefone }}</p>
        <p><strong>Email:</strong> {{ cliente.email }}</p>
        <p><strong>Endereço:</strong> {{ cliente.endereco }}</p>
    </div>

```

```

</div>

<!-- Lista de animais -->
<div class="d-flex justify-content-between align-items-center mb-3">
  <h5 class="card-title mb-0">Pets</h5>
  <a href=" " class="btn btn-warning btn-sm">+ Adicionar Novo Anim
al</a>
</div>
{% if animais %}
<ul class="list-unstyled">
  {% for animal in animais %}
    <li class="list-group-item">
      <strong>{{ animal.nome }}</strong> / {{ animal.raca }}
      <button type="button" class="btn btn-info btn-sm select-animal
-btn" data-animal-id="{{ animal.id }}">Selecionar</button>
    </li>
  {% endfor %}
</ul>
{% else %}
  <p class="text-muted">Nenhum animal cadastrado para este cliente.
</p>
{% endif %}
<hr>

```

```

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

<script>
$(document).ready(function() {
  $('.select-animal-btn').click(function() {
    var animalId = $(this).data('animal-id');
    $('#id_animal').val(animalId); // Define o valor do campo oculto
    alert('Animal selecionado com ID: ' + animalId);
  });
});

```

```
});  
</script>
```

14. Adicionar pet, modal

```
# Formulário para o modelo Animal  
class AnimalForm(forms.ModelForm):  
    class Meta:  
        model = Animal  
        fields = ['nome', 'especie', 'raca', 'idade', 'peso']  
        widgets = {  
            'nome': forms.TextInput(attrs={'class': 'form-control'}),  
            'idade': forms.NumberInput(attrs={'class': 'form-control'}),  
            'especie': forms.Select(attrs={'class': 'form-select'}),  
            'raca': forms.TextInput(attrs={'class': 'form-control'}),  
            'peso': forms.NumberInput(attrs={'class': 'form-control'}),  
        }
```

views.py

```
# views.py  
from django.http import JsonResponse  
from django.shortcuts import render  
from .forms import AnimalForm  
  
def add_animal(request):  
    if request.method == 'POST':  
        form = AnimalForm(request.POST)  
        if form.is_valid():  
            animal = form.save(commit=False)  
            cpf = request.POST.get("cpf")  
            print(cpf)  
            animal.dono = Cliente.objects.get(cpf=cpf)  
            animal.save()
```

```

        return JsonResponse({'id': animal.id, 'nome': animal.nome})
    else:
        return JsonResponse({'errors': form.errors}, status=400)
    else:
        form = AnimalForm()
    return render(request, 'add_animal_modal.html', {'form': form})

```

add_animal_modal.html

```

<!-- add_animal_modal.html -->
<div class="modal fade" id="animalModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content" id="modalContent">
      <div class="modal-header">
        <h5 class="modal-title">Add Pet</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" a
ria-label="Close"></button>
      </div>
      <div class="modal-body">
        <form id="animalForm">
          {% csrf_token %}
          {{ form_pet.as_p }}
          <input type="hidden" name="cpf" value="{{ cliente.cpf }}">
          <button type="submit" class="btn btn-primary">Salvar</button>
        </form>
      </div>
    </div>
  </div>
</div>
</div>
</div>

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

<script>
$( '#animalForm' ).submit(function(e){
  e.preventDefault();
  $.ajax({

```

```

url: "/add_animal/",
method: "POST",
data: $(this).serialize(),
success: function(data){
    // fecha modal
    $('#animalModal').modal('hide');
    // adiciona novo animal na lista
    let html_option = `<li class="list-group-item">
        <strong>${data.nome}</strong>
        <button type="button" class="btn btn-info btn-sm select-animal
-btn" data-animal-id="${data.id}">Selecionar</button>
    </li>`;
    $('#animalList').append(html_option);

    let newOption = new Option(data.nome, data.id, true, true);
    $('#id_animal').append(newOption).trigger('change');
},
error: function(xhr){
    alert('Erro ao adicionar animal');
}
});
});
</script>

```

agendar_consulta.html

```

<button type="button" class="btn btn-sm btn-success" data-bs-toggle
="modal" data-bs-target="#animalModal">
    + Animal
</button>

{% include 1add_animal_modal.html %}

<script>
$('#animalModal').on('show.bs.modal', function () {

```

```

    $('#modalContent').load("{% url 'add_animal' %}");
});
</script>

```

15. Adicionar cliente

forms.py

```

# Formulário para o modelo Clientes
class ClienteForm(forms.ModelForm):
    class Meta:
        model = Cliente
        fields = ['nome', 'cpf', 'telefone', 'email', 'endereco']

    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)

        for field in self.fields.values():
            field.widget.attrs.update({'class': 'form-control'})

```

views.py

```

def add_cliente(request):
    if request.method == 'POST':
        form = ClienteForm(request.POST)
        if form.is_valid():
            cliente = form.save()
            return JsonResponse(
                {'redirect_url': f'/agendar_consulta/?cpf={cliente.cpf}'}
            )
        else:
            return JsonResponse({'errors': form.errors}, status=400)
    else:
        form = ClienteForm()
        return render(request, 'add_cliente_modal.html', {'form': form})

```

add_cliente_modal.html

```

<div class="modal fade" id="clienteModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">

      <div class="modal-header">
        <h5 class="modal-title">Add Cliente</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>

      <div class="modal-body">
        <form id="clienteForm">
          {% csrf_token %}
          {{ form_cliente.as_p }}
          <button type="submit" class="btn btn-primary">Salvar</button>
        </form>
      </div>
    </div>
  </div>
</div>

```

agendar_consulta.html

```

<script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>

<script>
$(document).on("submit", "#clienteForm", function(e){
  e.preventDefault();

  $.ajax({
    url: "/add_cliente/",
    method: "POST",
    data: $(this).serialize(),

```

```

    success: function(response){
        $("#clienteModal").modal("hide");

        $("#cpfInput").val(response.cpf);

        window.location.href = response.redirect_url;
    },
    error: function(xhr){
        alert("Erro: " + xhr.responseJSON.errors);
    }
});
});
</script>

```

no agendamento

```

<button type="button" class="btn btn-success" data-bs-toggle="modal"
data-bs-target="#clienteModal">
    Adicionar Cliente
</button>

```

16. Vamo tentar colocar um calendario simples

<https://fullcalendar.io/docs>

views.py

```

def consulta_eventos(request):
    eventos = []
    for c in Consulta.objects.all():
        eventos.append({
            "title": f"{c.animal.nome} - {c.veterinario.nome}",
            "start": c.data.strftime("%Y-%m-%dT%H:%M:%S"),
        })

```

```
# print(eventos)
return JsonResponse(eventos, safe=False)
```

lista_consulta.html

```
<script src='https://cdn.jsdelivr.net/npm/fullcalendar@6.1.19/index.global.min.js'></script>
<script src="https://cdn.jsdelivr.net/npm/@fullcalendar/core@5.9.0/locales-all.global.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/@fullcalendar/daygrid@6.1.19/index.global.min.js"></script>

<div id="calendar"></div>

<script>
document.addEventListener('DOMContentLoaded', function() {
  var calendarEl = document.getElementById('calendar');
  var calendar = new FullCalendar.Calendar(calendarEl, {
    initialView: 'dayGridMonth',
    locale: 'pt-br',
    events: '/eventos/', // sua rota Django que retorna JSON
  });

  calendar.render();
});
</script>
```

configuração (fullcalendar)

```
// cabeçalho
headerToolbar: {
  left: 'prev,next today',
  center: 'title',
  right: 'dayGridMonth,timeGridWeek,timeGridDay,listWeek'
```

```

},

dateClick: function(info) {
  alert("Você clicou em: " + info.dateStr);
}

eventClick: function(info) {
  alert("Evento: " + info.event.title);
}

```

cor aleatoria

```
import random
```

```
"color": "#" + "".join([random.choice("0123456789ABCDEF") for _ in range(6)])
```

17. login

```

<div class="d-flex justify-content-center align-items-center">
  <div class="card shadow p-4" style="max-width: 400px; width: 100%;">
    <h2 class="text-center mb-4">Login</h2>

    {% if form.errors %}
      <div class="alert alert-danger text-center">
        Usuário ou senha incorretos
      </div>
    {% endif %}

    <form method="post">
      {% csrf_token %}

      <div class="mb-3">

```

```

        <label for="id_username" class="form-label">Usuário</label>
        <input type="text" name="username" id="id_username" class="form
-control" required>
    </div>

    <div class="mb-3">
        <label for="id_password" class="form-label">Senha</label>
        <input type="password" name="password" id="id_password" class
="form-control" required>
    </div>

    <button type="submit" class="btn btn-primary w-100">Entrar</button
>
    </form>
</div>
</div>

```

views.py

```

from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login
from django.contrib.auth.forms import AuthenticationForm

def login_view(request):
    if request.method == "POST":
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect('home') # troque para sua URL de destino
    else:
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})

```

```
from django.contrib.auth.decorators import login_required

@login_required(login_url='login')
```

Logout

```
@login_required(login_url='login')
def logout_view(request):
    logout(request)
    return redirect('login')
```

base.html

```
<div class="d-flex gap-3 align-content-center justify-content-end m-2">
    {% if request.user.is_authenticated %}
    <a class="btn btn-transparent">Olá, {{ request.user.username }}!</a>
    >
    <a class="btn btn-danger" href="{% url 'logout' %}">Logout</a>
    {% endif %}
</div>
```

18. Melhorada

Lista de Animais pode ser um Card

```
<h1 class="mb-4">Lista de Animais</h1>
<div class="row">
    {% for animal in animais %}
    <div class="col-md-4 mb-3">
        <div class="card">
            
```

```

<div class="card-body">
  <h5 class="card-title">{{ animal.nome }}</h5>
  <p class="card-text">
    Espécie: {{ animal.get_especie_display }}<br>
    Raça: {{ animal.raca }}
  </p>
  <p>
    Dono: {{ animal.dono.nome }}
  </p>
</div>
</div>
</div>
{% empty %}
<div class="col-12 text-center">
  Nenhum animal cadastrado.
</div>
{% endfor %}
</div>

```

inicio.html

```

<div class="text-center my-5">
  <h1 class="display-4 fw-bold">Clínica do Bicho</h1>
  <p class="lead text-muted">Bem-vindo à Clínica do Bicho! Escolha uma
  opção abaixo para começar.</p>
</div>

<div class="d-flex flex-row gap-4 justify-content-center align-items-center flex-wrap">
  <div class="text-center">
    
  </div>

  <div class="d-flex flex-column justify-content-center gap-3">

```

```

<div class="card text-center shadow p-4 w-md-25 border-primary h
over-scale" style="width: 350px; transition: transform 0.2s;">
  <a href="{% url 'lista_consultas' %}" class="btn btn-transparent te
xt-primary w-100 py-3 fw-bold">
    Lista Consultas
  </a>
</div>
<div class="card text-center shadow p-4 w-md-25 border-primary h
over-scale" style="width: 350px; transition: transform 0.2s;">
  <a href="#" class="btn btn-transparent text-primary w-100 py-3 f
w-bold">
    Veterinários
  </a>
</div>
<div class="card text-center shadow p-4 w-md-25 border-primary h
over-scale" style="width: 350px; transition: transform 0.2s;">
  <a href="{% url 'lista_animais' %}" class="btn btn-transparent text
-primary w-100 py-3 fw-bold">
    Lista Animais
  </a>
</div>
</div>
</div>

<style>
.hover-scale:hover {
  transform: scale(1.05);
}
</style>

```

19. Data Disponível para consulta com base no medico.

Formulário de ser escrito de varias formas.

```

<form method="POST">
  {% csrf_token %}

  <div class="mb-3">
    <label for="animal">Animal</label>
    {{ form.animal }}
  </div>
  <hr>

  <div class="row">
    <div class="col-md-6 mb-3">
      <label for="veterinario">Veterinário</label>
      {{ form.veterinario }}
    </div>
    <div class="col-md-6 mb-3">
      <div class="d-flex">
        <button type="button" class="btn btn-secondary btn-sm" data-
a-bs-toggle="modal" data-bs-target="#dataDoctorModal">Selecionar Da
ta Disponível</button>

        <div class="ms-2">
          <label for="data">Data</label>
          {{ form.data }}
        </div>
      </div>
    </div>
  </div>
  <hr>
  <div class="mb-3">
    <label for="motivo">Motivo</label>
    {{ form.motivo }}
  </div>

```

```

<div class="mb-3">
  <label for="observacoes">Observações</label>
  {{ form.observacoes }}
</div>

<!-- {{ form.as_p }} -->
<button type="submit" name="salvar" class="btn btn-primary">Agendar</button>
{% comment %} <a href="{% url 'lista_consultas' %}" class="btn btn-secondary">Cancelar</a> {% endcomment %}
</form>

```

```

<div class="modal modal-xl fade" id="dataDoctorModal" tabindex="-1">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">Ver data Disponível</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        Calendario aqui
      </div>
    </div>
  </div>
</div>

```

Feito isso já temos um norte.

Primeiro vamos criar uma função para puxar todas as consultas de um determinado veterinário.

views.py

```

@login_required(login_url='login')
def consulta_eventos_veterinario(request):
    veterinario = request.GET.get("veterinario")
    print(veterinario)
    eventos = []
    for c in Consulta.objects.filter(veterinario__id=veterinario):
        eventos.append({
            "id": c.id,
            "title": f"{c.animal.nome} - {c.veterinario.nome}",
            "start": c.data.strftime("%Y-%m-%dT%H:%M:%S"),
            "color": "green"
        })
    return JsonResponse(eventos, safe=False)

```

agendar_consulta.html

```

<script>
    let calendar_doctor;

    // Função simplificada para inicializar o calendário
    function initializeCalendar() {

        // Obter o ID do veterinário selecionado
        var veterinarioSelect = document.querySelector('[name="veterinario"]');
        var veterinariold = veterinarioSelect ? veterinarioSelect.value : null;

        if (!veterinariold) {
            alert('Por favor, selecione um veterinário primeiro.');
            return;
        }

        var calenderDoctor = document.getElementById('calendar_doctor');
```

```

// Limpar calendário existente
if (calendar_doctor) {
    calendar_doctor.destroy();
}

calendar_doctor = new FullCalendar.Calendar(calenderDoctor, {
    initialView: 'dayGridMonth',
    locale: 'pt-br',
    events: '/eventos_doctor/?veterinario=' + veterinariold,

    headerToolbar: {
        left: 'prev,next today',
        center: 'title',
        right: 'dayGridMonth,timeGridWeek,timeGridDay,listWeek'
    },

    // Bloquear datas passadas
    // validRange: {
    //     start: new Date().toISOString().split('T')[0] // Data atual em forma
to YYYY-MM-DD
    // },

    dateClick: function(info) {
        alert("Você clicou em: " + info.dateStr);
    },

    eventClick: function(info) {
        alert("Evento: " + info.event.title);
    }

});

// Forçar o renderizado corretamente
setTimeout(function() {
    calendar_doctor.render();
}, 50);

```

```

}

// Mostra veterinário selecionado
document.addEventListener('DOMContentLoaded', function() {
  $(document).on('change', '[name="veterinario"]', function() {
    console.log('Veterinário mudou', $(this).val());
  });
});

// Inicializar calendário quando o modal é aberto
$('#dataDoctorModal').on('shown.bs.modal', function () {
  setTimeout(function() {
    initializeCalendar();
  }, 200);
});

</script>

```

(Melhoria)

A gente pode controlar data disponível de cada veterinario via db ou podemos fazer isso hardcore. fixa no script do fullcalendar mesmo por enquanto só pra entender como funciona.

html

```

initialView: 'timeGridWeek', // New

// Configurações de horário
slotMinTime: '08:00:00',
slotMaxTime: '17:00:00',
slotDuration: '01:00:00',
allDaySlot: false,

```

```

// Horário de trabalho (excluindo almoço)
businessHours: [
  { daysOfWeek: [1, 2, 3, 4, 5], startTime: '08:00', endTime: '12:00' },
  { daysOfWeek: [1, 2, 3, 4, 5], startTime: '13:00', endTime: '17:00' }
],

// Manipuladores de eventos simplificados
dateClick: function(info) {
  // Verificar se é horário de almoço
  const hour = new Date(info.date).getHours();
  if (hour === 12) {
    alert("Horário de almoço não disponível.");
    return;
  }

  // Selecionar a data
  const dataFormatada = info.date.toISOString().slice(0, 16).replace('T', ' ');
  document.querySelector('[name="data"]').value = dataFormatada;
  $('#dataDoctorModal').modal('hide');
},

eventClick: function(info) {
  if (info.event.title === "Disponível") {
    // Selecionar horário disponível
    const dataFormatada = info.event.start.toISOString().slice(0, 16).replace('T', ' ');
    document.querySelector('[name="data"]').value = dataFormatada;
    $('#dataDoctorModal').modal('hide');
  } else {
    alert("Consulta já agendada: " + info.event.title);
  }
}
}

```

views.py

```

@login_required(login_url="login")
def consulta_eventos_veterinario(request):
    from datetime import datetime, timedelta

    veterinario_id = request.GET.get("veterinario")
    if not veterinario_id:
        return JsonResponse([], safe=False)

    # Consultas agendadas já no formato necessário
    consultas = (
        Consulta.objects.filter(veterinario_id=veterinario_id)
        .select_related("animal", "veterinario")
    )

    consultas_dict = {
        c.data.strftime("%Y-%m-%d-%H"): c.id for c in consultas
    }

    eventos = [
        {
            "id": c.id,
            "title": f"{c.animal.nome} - {c.veterinario.nome}",
            "start": c.data.strftime("%Y-%m-%dT%H:%M:%S"),
            "color": "#dc3545", # vermelho
        }
        for c in consultas
    ]

    hoje = datetime.now().date()
    horarios = [8, 9, 10, 11, 13, 14, 15, 16]

    for i in range(7):
        data = hoje + timedelta(days=i)
        if data.weekday() >= 5:
            continue

```

```

for hora in horarios:
    key = f"{data.strftime('%Y-%m-%d')}-{hora}"
    if key not in consultas_dict:
        slot_inicio = datetime.combine(data, datetime.min.time().replace
(hour=hora))
        eventos.append(
            {
                "id": f"disp-{data}-{hora}",
                "title": "Disponível",
                "start": slot_inicio.strftime("%Y-%m-%dT%H:%M:%S"),
                "color": "#28a745", # verde
            }
        )
return JsonResponse(eventos, safe=False)

```

▼ Django API

1. Instalar DRF

```

pip install djangorestframework
pip install djangorestframework-simplejwt

```

No `settings.py` :

```

INSTALLED_APPS = [
    ...,
    'rest_framework',
    'rest_framework.authtoken',
    'core',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (

```

```
    'rest_framework_simplejwt.authentication.JWTAuthentication',  
    )  
}
```

Configurar CORS.

serve para dizer **quais domínios externos podem nossa API**.

Sem ele, o navegador bloqueia requisições de origem diferente

(ex: Angular → Django).

resumo libera ou restringe quem pode consumir sua API via browser. hehe

```
pip install django-cors-headers
```

1. Adicione no `settings.py` :

```
INSTALLED_APPS = [  
    ...,  
    'corsheaders',  
]  
  
MIDDLEWARE = [  
    'corsheaders.middleware.CorsMiddleware', # Colocar no topo  
    ...,  
]  
  
# Permitir todas as origens (para teste)  
CORS_ALLOW_ALL_ORIGINS = True  
  
# Ou, para mais segurança, permitir apenas seu app:  
# CORS_ALLOWED_ORIGINS = [  
#     "http://localhost:8100",
```

```
# "http://localhost:4200",  
# ]
```

2. Criar **serializers** (`core/serializers.py`)

```
from rest_framework import serializers  
from .models import Cliente, Animal, MedicoVeterinario, Consulta  
  
class ClienteSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Cliente  
        fields = '__all__'  
  
class AnimalSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Animal  
        fields = '__all__'  
  
class MedicoVeterinarioSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = MedicoVeterinario  
        fields = '__all__'  
  
class ConsultaSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = Consulta  
        fields = '__all__'
```

3. Criar **views** (`core/views.py`)

```
from rest_framework import viewsets  
from .models import Cliente, Animal, MedicoVeterinario, Consulta  
from .serializers import ClienteSerializer, AnimalSerializer, MedicoVeterina
```

rioSerializer, ConsultaSerializer

```
class ClienteViewSet(viewsets.ModelViewSet):  
    queryset = Cliente.objects.all()  
    serializer_class = ClienteSerializer
```

```
class AnimalViewSet(viewsets.ModelViewSet):  
    queryset = Animal.objects.all()  
    serializer_class = AnimalSerializer
```

```
class MedicoVeterinarioViewSet(viewsets.ModelViewSet):  
    queryset = MedicoVeterinario.objects.all()  
    serializer_class = MedicoVeterinarioSerializer
```

```
class ConsultaViewSet(viewsets.ModelViewSet):  
    queryset = Consulta.objects.all()  
    serializer_class = ConsultaSerializer
```

4. Criar endpoint de login com JWT

`core/views.py` :

```
from rest_framework_simplejwt.views import TokenObtainPairView  
from rest_framework_simplejwt.serializers import TokenObtainPairSerializ  
er  
  
class MyTokenObtainPairSerializer(TokenObtainPairSerializer):  
    @classmethod  
    def get_token(cls, user):  
        token = super().get_token(user)  
        # Adicionar info extra no token se quiser  
        token['username'] = user.username  
        return token
```

```
class MyTokenObtainPairView(TokenObtainPairView):
    serializer_class = MyTokenObtainPairSerializer
```

4. Criar **rotas API** (`clinica_veterinaria/urls.py`)

```
from django.contrib import admin
from django.urls import path, include
from core.views import MyTokenObtainPairView
from rest_framework import routers
from core.views import ClienteViewSet, AnimalViewSet, MedicoVeterinarioViewSet, ConsultaViewSet

router = routers.DefaultRouter()
router.register(r'clientes', ClienteViewSet)
router.register(r'animais', AnimalViewSet)
router.register(r'veterinarios', MedicoVeterinarioViewSet)
router.register(r'consultas', ConsultaViewSet)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include(router.urls)),

    path('api/login/', MyTokenObtainPairView.as_view(), name='token_obtain_pair'),
]
```

5. Testar a API

```
python manage.py runserver
```

Abra:

- `http://127.0.0.1:8000/api/clientes/`
- `http://127.0.0.1:8000/api/animais/`
- `http://127.0.0.1:8000/api/consultas/`

```
curl -X POST http://127.0.0.1:8000/api/login/ \
-H "Content-Type: application/json" \
-d '{
  "username": "seu_usuario",
  "password": "sua_senha"
}'
```

▼ Ionic APP

APP simples, Angular + Ionic

1. Login
2. Listar suas consultas
3. Agendar consulta

▼ CreateApp, Servicos API, Login (teste)

Ionic Angular

```
npm install -g @ionic/cli
ionic start ClinicaDoBichoClient blank --type=angular
cd ClinicaDoBichoClient
```

Instalar dependências

```
npm install @angular/common@latest
npm install @angular/forms@latest
npm install @ionic/storage-angular
npm install @capacitor/core @capacitor/cli
```

service de API

`src/app/services/api.service.ts`

```
ionic generate service services/api
```

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Consulta {
  id?: number;
  animal: number;
  veterinario: number;
  data: string;
  motivo: string;
  observacoes?: string;
}

@Injectable({
  providedIn: 'root' // já disponível em todo app
})
export class ApiService {
  private baseUrl = 'http://127.0.0.1:8000/api'; // backend Django
```

```

constructor(private http: HttpClient) {}

// login retorna login
login(username: string, password: string): Observable<any> {
  return this.http.post(`${this.baseUrl}/login/`, { username, password
});
}

// pegar consultas
getConsultas(): Observable<Consulta[]> {
  return this.http.get<Consulta[]>(`${this.baseUrl}/consultas/`);
}

// criar consulta
agendarConsulta(consulta: Consulta): Observable<Consulta> {
  return this.http.post<Consulta>(`${this.baseUrl}/consultas/`, consult
a);
}
}

```

No Angular, o **HttpClient** retorna sempre um **Observable**.

- Ele representa **requisições assíncronas** (que podem demorar, como chamadas de API).
- Assim, você pode **inscrever-se** (**.subscribe(...)**) para tratar a resposta **quando ela chegar**.

Exemplo:

```

this.apiService.login(this.username, this.password).subscribe({
  next: (res) => {
    console.log("Token recebido:", res.access);
    if (res.access) {
      this.storage.set('token', res.access).then(() => {
        this.router.navigate(['/consultas']); // só navega depois de salvar
      });
    }
  }
});

```

```

    }
  });
},
error: (err) => {
  console.error("Erro no login:", err);
  alert('Usuário ou senha inválidos!');
}
});

```

página de Login

ionic generate page login

login.page.ts :

```

import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Storage } from '@ionic/storage-angular';

import {
  IonContent,
  IonHeader,
  IonTitle,
  IonToolbar } from '@ionic/angular/standalone';

import { ApiService } from '../services/api';
import { Router } from '@angular/router';

@Component({
  selector: 'app-login',
  templateUrl: './login.page.html',
  styleUrls: ['./login.page.scss'],
  standalone: true,

```

```

    imports: [IonContent, IonHeader, IonTitle, IonToolbar, CommonModule,
FormsModule]
  })
  export class LoginPage {

    username = '';
    password = '';

    constructor(
      private api: ApiService,
      private router: Router,
      private storage: Storage
    ) {}

    login() {
      this.api.login(this.username, this.password).subscribe({
        next: (res) => {
          if (res.access) {
            this.storage.set('token', res.access).then(() => {
              this.router.navigate(['/consultas']);
            });
          }
        },
        error: (err) => {
          console.error("Erro no login:", err);
          alert('Usuário ou senha inválidos!');
        }
      });
    }
  }
}

```

login.page.html :

```

<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Login</ion-title>
  </ion-toolbar>
</ion-header>

```

```

</ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <form (ngSubmit)="login()" class="login-form">

    <ion-item>
      <ion-label position="floating">Usuário</ion-label>
      <ion-input [(ngModel)]="username" name="username" required>
    </ion-input>
    </ion-item>

    <ion-item>
      <ion-label position="floating">Senha</ion-label>
      <ion-input [(ngModel)]="password" name="password" type="password" required></ion-input>
    </ion-item>

    <ion-button expand="block" type="submit" class="login-button">
      Entrar
    </ion-button>

  </form>
</ion-content>

.login-form {
  display: flex;
  flex-direction: column;
  gap: 16px;
  margin-top: 40px;
}

.login-button {
  margin-top: 20px;
}

```

<https://angular.dev/errors/NG01203>

<https://forum.ionicframework.com/t/ngmodel-binding-ionic-7/232390>

<https://stackoverflow.com/questions/60814042/error-no-value-accessor-for-form-control-with-name-with-ionic-5-and-angular-9>

Error storage.

<https://forum.ionicframework.com/t/ionicstorage-for-angular-in-ionic-7/232596>

main.ts

```
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withPreloading, PreloadAllModules } from '@angular/router';
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular/standalone';
import { provideHttpClient } from '@angular/common/http';
import { importProvidersFrom } from '@angular/core'; // Importe esta função

import { AppComponent } from './app/app.component';
import { routes } from './app/app.routes';

// Importe o módulo do Ionic Storage
import { IonicStorageModule } from '@ionic/storage-angular';

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules)),
    provideHttpClient(),
    importProvidersFrom(IonicStorageModule.forRoot())
  ]
});
```

```
],  
});
```

```
import { Injectable } from '@angular/core';  
import { Storage } from '@ionic/storage-angular';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class StorageService {  
  private _storage: Storage | null = null;  
  
  // Injetamos o Storage "cru" do Ionic  
  constructor(private storage: Storage) {  
    // Iniciamos a inicialização no construtor do serviço  
    this.init();  
  }  
  
  // O método de inicialização que lida com a parte assíncrona  
  async init() {  
    // Se o storage já foi inicializado, não faz nada.  
    if (this._storage) {  
      return;  
    }  
    // Chama o create() e, quando terminar, atribui a instância pronta à  
    // nossa variável.  
    const storage = await this.storage.create();  
    this._storage = storage;  
  }  
  
  // Criamos nossos próprios métodos 'set' e 'get' que garantem que o  
  // storage está pronto.  
  public async set(key: string, value: any) {  
    // Espera a inicialização terminar, caso ainda não tenha terminado.  
    await this.init();
```

```

    return this._storage?.set(key, value);
  }

  public async get(key: string) {
    await this.init();
    return this._storage?.get(key);
  }

  public async remove(key: string) {
    await this.init();
    return this._storage?.remove(key);
  }
}

```

▼ CreateComponent, Consumindo API, Lista e Agendar Consulta

listar consultas

ionic generate page consultas

consultas.page.ts :

```

import { Component, OnInit } from '@angular/core';
import { ApiService, Consulta } from '../services/api.service';
import { IonicModule } from '@ionic/angular';
import { CommonModule } from '@angular/common';

@Component({
  standalone: true,
  selector: 'app-consultas',
  templateUrl: './consultas.page.html',
  styleUrls: ['./consultas.page.scss'],
  imports: [CommonModule, IonicModule]
})
export class ConsultasPage implements OnInit {
  consultas: Consulta[] = [];
}

```

```

constructor(private api: ApiService) {}

ngOnInit() {
  this.api.getConsultas().subscribe(data => this.consultas = data);
}
}

```

consultas.page.html :

```

<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Consultas</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-list>
    <ion-card *ngFor="let consulta of consultas" class="consulta-card">
      <ion-card-header>
        <ion-card-subtitle>{{ consulta.data | date:'short' }}</ion-card-subtitle>
        <ion-card-title>{{ consulta.paciente }}</ion-card-title>
      </ion-card-header>

      <ion-card-content>
        <p>Médico: {{ consulta.medico }}</p>
        <p>Status: {{ consulta.status }}</p>
      </ion-card-content>
    </ion-card>
  </ion-list>
</ion-content>

.consulta-card {
  border-radius: 12px;

```

```
margin-bottom: 12px;
}
```

Agendar Consulta

```
ionic generate page agendar-consulta
```

agendar-consulta.page.ts :

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { IonContent, IonHeader, IonTitle, IonToolbar } from '@ionic/angular/standalone';
import { ApiService, Consulta } from '../services/api';
import { Router } from '@angular/router';

@Component({
  selector: 'app-agendar-consulta',
  templateUrl: './agendar-consulta.page.html',
  styleUrls: ['./agendar-consulta.page.scss'],
  standalone: true,
  imports: [IonContent, IonHeader, IonTitle, IonToolbar, CommonModule, FormsModule]
})
export class AgendarConsultaPage {

  consulta?: Consulta;

  constructor(private api: ApiService, private router: Router) {}

  OnInit() {}

  agendar() {
    this.api.agendarConsulta(this.consulta!).subscribe({
      next: (data) => {
```

```

        console.log("Status:", data);
        this.router.navigate(['/consultas']);
    },
    error: (error) => {
        console.log("Status:", error);
        alert('Erro ao agendar!')
    }
  });
}

}

```

agendar-consulta.page.html :

```

<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Agendar Consulta</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding" fullscreen>

  <form (ngSubmit)="agendar()" class="agendar-form">

    <ion-item>
      <ion-label position="floating">ID do Animal</ion-label>
      <ion-input type="number" [(ngModel)]="consulta.animal" name
      ="animal" required></ion-input>
    </ion-item>

    <ion-item>
      <ion-label position="floating">ID do Veterinário</ion-label>
      <ion-input type="number" [(ngModel)]="consulta.veterinario" nam
      e="veterinario" required></ion-input>
    </ion-item>
  </form>

```

```

<ion-item>
  <ion-label position="floating">Data e Hora</ion-label>
  <ion-datetime
    [(ngModel)]="consulta.data"
    name="data"
    display-format="DD/MM/YYYY HH:mm"
    picker-format="DD/MM/YYYY HH:mm"
    required>
  </ion-datetime>
</ion-item>

<ion-item>
  <ion-label position="floating">Motivo</ion-label>
  <ion-textarea [(ngModel)]="consulta.motivo" name="motivo" required>
</ion-textarea>
</ion-item>

<ion-button expand="block" color="success" shape="round" type="submit">
  <ion-icon name="checkmark-circle-outline" slot="start"></ion-icon>
  Agendar
</ion-button>

</form>

</ion-content>

.agendar-form {
  display: flex;
  flex-direction: column;
  gap: 16px;
  margin-top: 20px;
}

```

▼ Limitar data

min e max de hoje:

```
<ion-item>
  <ion-label position="floating">Data e Hora</ion-label>
  <ion-datetime
    [(ngModel)]="consulta.data"
    name="data"
    display-format="DD/MM/YYYY HH:mm"
    picker-format="DD/MM/YYYY HH:mm"
    [min]="minDate"
    [max]="maxDate"
    required>
  </ion-datetime>
</ion-item>
```

E no `component.ts`:

```
// Limita de hoje até 30 dias à frente exceto hora
minDate = new Date().toISOString();
maxDate = new Date(new Date().setDate(new Date().getDate() +
30)).toISOString();

// no submit a hora
// validação simples do horário: 08:00-12:00 ou 13:00-17:00
const date = new Date(this.consulta.data);
const hour = date.getHours();
if (!(hour >= 8 && hour < 12) || (hour >= 13 && hour < 17)) {
  alert('Horário deve ser entre 08:00-12:00 ou 13:00-17:00');
  return;
}
```

Forms SELECT

```
<ion-item>
  <ion-label position="floating">Animal</ion-label>
  <!-- <ion-input [(ngModel)]="consulta.animal.nome" name="animal
  I" required></ion-input> →

  <ion-select [(ngModel)]="consulta.animal" name="animal" require
  d>
    @for (animal of animais; track animal.id) {
      <ion-select-option [value]="animal.id">
        {{ animal.nome }}
      </ion-select-option>
    }
  </ion-select>

</ion-item>

<ion-item>
  <ion-label position="floating">Veterinário</ion-label>
  <!-- <ion-input [(ngModel)]="consulta.veterinario.nome" name="v
  eterinario" required></ion-input> →

  <ion-select [(ngModel)]="consulta.veterinario" name="veterinari
  o" required>
    @for (veterinario of veterinarios; track veterinario.id) {
      <ion-select-option [value]="veterinario.id">
        {{ veterinario.nome }}
      </ion-select-option>
    }
  </ion-select>

</ion-item>
```

Lista de Animais e Veterinarios

```
animais: Animal[] = [];  
veterinarios: Veterinario[] = [];  
  
listAnimais() {  
  this.api.listAnimais().subscribe({  
    next: (data) => {  
      console.log("Lista de Animais:", data);  
      this.animais = data;  
    },  
    error: (error) => {  
      console.error('Erro ao buscar animais:', error);  
    }  
  });  
}  
  
listVeterinarios() {  
  this.api.listVeterinarios().subscribe({  
    next: (data) => {  
      console.log("Lista de Veterinários:", data);  
      this.veterinarios = data;  
    },  
    error: (error) => {  
      console.error('Erro ao buscar veterinários:', error);  
    }  
  });  
}
```

Altera View para agendar POST

```
class ConsultaSerializerAdd(serializers.ModelSerializer):
```

```

animal = serializers.PrimaryKeyRelatedField(
    queryset=Animal.objects.all(), write_only=True
)
veterinario = serializers.PrimaryKeyRelatedField(
    queryset=MedicoVeterinario.objects.all(), write_only=True
)

class Meta:
    model = Consulta
    fields = '__all__'

```

```

def get_serializer_class(self):
    if self.action == 'create':
        return ConsultaSerializerAdd
    return ConsultaSerializer

```

▼ Login, auth, guard, inspetor, loading, Logout, html

AuthGuard: Protege as rotas, acesso somente quando faz o login

`src/app/auth/auth.guard.ts`

```

import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from '../services/auth-service';

@Injectable({
    providedIn: 'root'
})
export class AuthGuard implements CanActivate {

    constructor(
        private authService: AuthService,
        private router: Router
    ) {}

```

```

async canActivate(): Promise<boolean> {
  // const token = await this.storage.get('token'); ERRADO
  const token = await this.authService.getAccessToken();

  if (token) {
    return true; // Acesso liberado.
  } else {
    this.router.navigate(['/login']);
    return false;
  }
}

```

exemplo:

```

{
  path: 'consultas',
  loadComponent: () => import('./consultas/consultas.page').then(m =>
m.ConsultasPage
), canActivate: [AuthGuard]
},

```

▼ storage refatorado (opcional)

```

// src/app/services/storage.service.ts
import { Injectable } from '@angular/core';
import { Storage } from '@ionic/storage-angular';

@Injectable({ providedIn: 'root' })
export class StorageService {
  private _storage: Storage | null = null;
  private initPromise: Promise<void>;

  constructor(private storage: Storage) {

```

```
// Guarda a Promise da inicialização para garantir que todos esp  
erem por ela.
```

```
  this.initPromise = this.init();  
}
```

```
private async init(): Promise<void> {  
  const storage = await this.storage.create();  
  this._storage = storage;  
  console.log('[StorageService] Storage inicializado com sucess  
o.');
```

```
}  
  
public async set(key: string, value: any): Promise<any> {  
  await this.initPromise; // Garante que o storage está pronto  
  console.log('[StorageService] Salvando... Chave: ${key}, Valor:  
${value}');
```

```
  return await this._storage!.set(key, value);  
}  
  
public async get(key: string): Promise<any> {  
  await this.initPromise; // Garante que o storage está pronto  
  const value = await this._storage!.get(key);  
  console.log('[StorageService] Lendo... Chave: ${key}, Valor enc  
ontrado: ${value}');
```

AuthService (guardar tokens)

Podemos salvar o refresh token também, Para ter um controle melhor disso vamos criar um serviço para salvar esses tokens. Access e o Refresh. OK

```
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';
```

```

import { StorageService } from './storage';

@Injectable({ providedIn: 'root' })
export class AuthService {

    private apiUrl = 'http://127.0.0.1:8000/api';
    private accessTokenKey = 'access_token';
    private refreshTokenKey = 'refresh_token';

    constructor(private http: HttpClient, private storage: StorageService)
    {}

    // Pega token
    async getAccessToken() {
        return await this.storage.get(this.accessTokenKey);
    }

    // Salva tokens
    async setTokens(access: string, refresh: string) {
        await this.storage.set(this.accessTokenKey, access);
        await this.storage.set(this.refreshTokenKey, refresh);
    }

    // Renova access token usando refresh token
    async refreshToken() {
        const refresh = await this.storage.get(this.refreshTokenKey);
        return this.http.post<any>(`${this.apiUrl}/auth/refresh/`, { refresh });
    }

    // Logout deixa ele aqui depois vamos usar.
    async logout() {
        await this.storage.remove(this.accessTokenKey);
        await this.storage.remove(this.refreshTokenKey);
    }
}

```

```

from django.urls import path
from rest_framework_simplejwt.views import (
    TokenRefreshView,
)

class MyTokenRefreshView(TokenRefreshView):
    pass

path('api/auth/refresh/', MyTokenRefreshView.as_view(), name='token_refresh'),

```

Interceptor: Garante que **todas as requisições automaticamente vão autenticadas com o token JWT** (se existir). Todas as requisições já vão com `Authorization: Bearer <token>` automático

`src/app/auth/token.interceptor.ts`

<https://www.learnrxjs.io/learn-rxjs/operators/creation/from>

```

import { HttpInterceptorFn, HttpResponse } from '@angular/common/http';
import { inject } from '@angular/core';
import { from, throwError, Observable } from 'rxjs';
import { switchMap, catchError } from 'rxjs/operators';
import { AuthService } from '../services/auth-service';
import { Router } from '@angular/router';

export const tokenInterceptor: HttpInterceptorFn = (req, next) => {
    const authService = inject(AuthService);
    const router = inject(Router);

    // Se for rota de login ou refresh, ignora e segue em frente.
    if (req.url.includes('/login') || req.url.includes('/auth/refresh')) {
        return next(req);
    }

```

```

}

// Para todas as outras rotas, adiciona o token.
return from(authService.getAccessToken()).pipe(
  switchMap(accessToken => {
    // Se não houver token, não há o que fazer, apenas continue.
    if (!accessToken) {
      return next(req);
    }

    // Clona a requisição e adiciona o cabeçalho de autorização.
    const request = req.clone({
      setHeaders: { Authorization: `Bearer ${accessToken}` },
    });

    // Envia a requisição com o token.
    return next(request).pipe(
      catchError((error: HttpResponse) => {
        // Se o erro for 401 (Não Autorizado), o token pode ter expirado.
        if (error.status === 401) {
          console.log('[Interceptor] Erro 401! Token expirado. Tentando r
enovar...');

          return from(authService.refreshToken()).pipe(
            switchMap((res: any) => {
              console.log('[Interceptor] Token renovado com sucesso!', re
s);

              // Salva os novos tokens recebidos.
              // Precisamos retornar a Promise para garantir a ordem.
              return from(authService.setTokens(res.access, res.refresh)).
pipe(
                switchMap(() => {
                  // Clona a requisição *original* com o *novo* token de ace
sso.

                  const newRequest = req.clone({
                    setHeaders: { Authorization: `Bearer ${res.access}` }

```

```

    });

    console.log('[Interceptor] Reenviando requisição com o novo token...');
    return next(newRequest); // Reenvia a requisição e o fluxo continua.
  })
  );
}),
catchError(refreshError ⇒ {
  console.error('[Interceptor] Falha ao renovar o token. Fazendo o logout.', refreshError);

  // Se a renovação falhar (ex: refresh token também expirou),
  // desloga o usuário e o envia para a página de login.
  authService.logout();
  router.navigate(['/login']);

  // Retorna o erro para que a chamada original saiba que falhou.
  return throwError(() ⇒ refreshError);
})
);
}
// Se não for 401, apenas repassa o erro.
return throwError(() ⇒ error);
})
);
})
);
};

```

Registrar o Interceptor no `main.ts`

```

import { provideHttpClient, withInterceptors } from '@angular/common/http';

```

```

import { tokenInterceptor } from './app/auth/token.interceptor';

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },

    provideIonicAngular( ),
    provideRouter(routes, withPreloading(PreloadAllModules)),

    provideHttpClient(
      withInterceptors([tokenInterceptor])
    ),

    importProvidersFrom(IonicStorageModule.forRoot())
  ],
});

```

vai da erro tem que fazer algumas correções, login.ts e guard
login.ts

```

constructor(
  private apiService: ApiService, // O serviço que fala com a API
  private authService: AuthService, // O serviço que gerencia os tokens
  private router: Router
) {}

login() {
  console.log('--- PROCESSO DE LOGIN INICIADO ---');
  this.apiService.login(this.username, this.password).subscribe({
    next: async (resposta) => {
      console.log('[LoginPage] Resposta da API simulada recebida:', resposta);

      if (resposta && resposta.access) {

```

```

    await this.authService.setTokens(resposta.access, resposta.refresh);

    console.log('[LoginPage] Navegando para /consultas...');
    this.router.navigate(['/consultas']); // Descomente para navegar
de verdade
  }
},
error: (err) => {
  console.error('Algo deu errado na simulação', err);
},
});
}
}
}

```

login.html

<https://ionicframework.com/docs/api/grid>

```

<ion-content class="login-content" scroll-y="false">

  <ion-grid class="login-grid">
    <ion-row class="login-row">
      <ion-col size="12" class="login-col">

        <!-- Logo -->
        <div class="logo-container">
          
          <h2 class="clinic-name">Clínica Do Bico</h2>
        </div>

        <form (ngSubmit)="login()" class="login-form">

```

```

    <ion-item>
      <ion-icon name="person-outline" slot="start"></ion-icon>
      <ion-label position="floating">Usuário</ion-label>
      <ion-input [(ngModel)]="username" name="username" require
d></ion-input>
    </ion-item>

    <ion-item>
      <ion-icon name="lock-closed-outline" slot="start"></ion-icon
>
      <ion-label position="floating">Senha</ion-label>
      <ion-input [(ngModel)]="password" name="password" type
="password" required></ion-input>
    </ion-item>

    <ion-button expand="block" type="submit" class="login-butto
n">
      Entrar
    </ion-button>

  </form>

  <div class="login-footer">
    <p>@OpenCodgo</p>
  </div>

</ion-col>
</ion-row>
</ion-grid>

</ion-content>

```

```

.login-content {
  --padding-top: 0;

```

```
--padding-bottom: 0;
--padding-start: 0;
--padding-end: 0;
--overflow: hidden;
}

.login-grid {
  height: 100vh;
  padding: 0;
  margin: 0;
}

.login-row {
  height: 100%;
  display: flex;
  align-items: center;
  justify-content: center;
  margin: 0;
}

.login-col {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  padding: 20px;
  max-width: 400px;
}

.logo-container {
  text-align: center;
  margin-bottom: 30px;

  img {
    width: 100px;
    height: 100px;
```

```

    background: #fff;
    border-radius: 50%;
    border: 3px solid #60b12a;
    box-shadow: 0 4px 15px rgba(45, 80, 22, 0.3);
    padding: 5px;
    display: block;
    margin: 0 auto 15px auto;
  }
}

.clinic-name {
  color: #2d5016;
  font-size: 24px;
  font-weight: 600;
  margin: 0;
  text-align: center;
}

.login-form {
  width: 100%;
  max-width: 350px;
}

ion-item {
  --background: rgba(255, 255, 255, 0.9);
  --border-radius: 12px;
  --highlight-color-focused: #2d5016;
  --padding-start: 16px;
  --padding-end: 16px;
  border-radius: 12px;
  margin-bottom: 15px;
}

ion-icon {
  color: #2d5016;
  margin-right: 12px;

```

```
    font-size: 20px;
  }

  ion-label {
    color: #2d5016 !important;
    font-weight: 500;
  }

  .login-button {
    margin-top: 25px;
    --background: #59a128;
    --background-activated: #339b52;
    --border-radius: 12px;
    font-weight: 600;
    height: 50px;
    font-size: 16px;
    text-transform: uppercase;
  }

  .login-footer {
    text-align: center;
    font-size: 14px;
    margin-top: 25px;
    font-weight: 600;
    color: #666;
  }
```

Puxar o nome de usuario

```
npm install jwt-decode
```

```
// auth.service.ts
import { jwtDecode } from 'jwt-decode';
```

```
// ... dentro da classe AuthService

async getUserProfile(): Promise<any | null> {
  const token = await this.getAccessToken();
  if (!token) {
    return null;
  }
  // Decodifica o token para pegar as informações (payload)
  const decodedToken: any = jwtDecode(token);
  return { username: decodedToken.username }; // Retorna o usernam
e
}
```

ion-loading

<https://ionicframework.com/docs/api/loading>

```
import { LoadingController } from '@ionic/angular/standalone';
```

```
constructor(
  private api: ApiService,
  private router: Router,
  private storage: StorageService,
  private loadingCtrl: LoadingController
) {}
```

```
async login() {
  // Cria e exibe o loading
  const loading = await this.loadingCtrl.create({
    message: 'Entrando...',
    spinner: 'crescent',
    backdropDismiss: false
  });
}
```

```
await loading.present();
```

exemplo:

```
this.api.login(this.username, this.password).subscribe({
  next: async (res) => {
    await loading.dismiss(); // esconde o loading
    if (res.access) {
      await this.storage.set('token', res.access);
      this.router.navigate(['/consultas']);
    }
  },
  error: async (err) => {
    await loading.dismiss(); // esconde o loading
    alert('Usuário ou senha inválidos!');
  }
});
```

Logout

```
async logout() {
  await this.storage.remove(this.accessTokenKey);
  await this.storage.remove(this.refreshTokenKey);
}
```

O Problema: O Token Ainda é Válido: é que o JWT é **stateless** (sem estado). Isso significa que o servidor não guarda um registro de quais tokens ele emitiu.

Blacklist (Lista Negra) de Tokens

É aqui que entra a **blacklist**. A ideia é no servidor para registrar os tokens que foram invalidados *antes* de sua expiração

```
INSTALLED_APPS = [
```

```
# ...  
'rest_framework',  
'rest_framework_simplejwt',  
'rest_framework_simplejwt.token_blacklist',  
]
```

```
python manage.py migrate
```

```
# views.py  
from rest_framework.views import APIView  
from rest_framework.response import Response  
from rest_framework import status  
from rest_framework.permissions import IsAuthenticated  
from rest_framework_simplejwt.tokens import RefreshToken  
  
class LogoutView(APIView):  
    permission_classes = (IsAuthenticated,)  
  
    def post(self, request):  
        try:  
            refresh_token = request.data["refresh"]  
            token = RefreshToken(refresh_token)  
            token.blacklist() # Adiciona o refresh token à blacklist  
  
            return Response(status=status.HTTP_205_RESET_CONTENT)  
        except Exception as e:  
            return Response(status=status.HTTP_400_BAD_REQUEST)
```

```
path('api/auth/logout/', LogoutView.as_view(), name='token_logout'),
```

```
app do ion: auth.service
```

```

// NOVO MÉTODO DE LOGOUT
async logout() {
  const refreshToken = await this.getRefreshToken();

  if (refreshToken) {
    // Envia o refresh token para a blacklist no servidor
    this.http.post(`${this.apiUrl}/auth/logout/`, { refresh: refreshToken })
      .subscribe({
        next: () => {
          console.log('Token invalidado no servidor com sucesso.');
```

```
await this.storage.remove(this.accessTokenKey);
await this.storage.remove(this.refreshTokenKey);
}
```

Layout Tabs (Mobile)

```
ionic generate page tabs --no-interactive
```

Configurar as Rotas das Abas

`tabs.routes.ts` .

```
// tabs/tabs.routes.ts

import { Routes } from '@angular/router';
import { TabsPage } from './tabs.page';
import { AuthGuard } from '../auth/auth.guard';

export const routes: Routes = [
  {
    path: '',
    component: TabsPage,
    canActivate: [AuthGuard],
    children: [
      {
        path: 'inicio',
        loadChildren: () => import('../inicio/inicio.page').then(m => m.InicioPage)
      },
      {
        path: 'consultas',
        loadChildren: () => import('../consultas/consultas.page').then(m => m.ConsultasPage)
      },
      {
```

```

    path: 'agendar',
    loadComponent: () => import('./agendar-consulta/agendar-consul
ta.page').then(m => m.AgendarConsultaPage)
  },
  {
    path: '',
    redirectTo: '/tabs/inicio',
    pathMatch: 'full',
  },
],
},
{
  // Redirecionamento geral: se o usuário logado tentar ir para a raiz,
  mande para as abas
  path: '',
  redirectTo: '/tabs/inicio',
  pathMatch: 'full',
},
];

```

```

<!-- Em src/app/tabs/tabs.page.html →
<ion-tabs>
  <ion-tab-bar slot="bottom">

    <ion-tab-button tab="inicio" href="/tabs/inicio">
      <ion-icon name="home-outline"></ion-icon>
      <ion-label>Início</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="consultas" href="/tabs/consultas">
      <ion-icon name="calendar-outline"></ion-icon>
      <ion-label>Consultas</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="agendar" href="/tabs/agendar">

```

```

    <ion-icon name="add-circle-outline"></ion-icon>
    <ion-label>Agendar</ion-label>
  </ion-tab-button>

</ion-tab-bar>
</ion-tabs>

```

```
ionic generate page inicio --no-interactive
```

```

import { Component, OnInit } from '@angular/core';
import { AuthService } from '../auth/auth.service';

@Component({
  selector: 'app-inicio',
  templateUrl: './inicio.page.html',
  styleUrls: ['./inicio.page.scss'],
})
export class InicioPage implements OnInit {
  username: string | null = null;

  constructor(private authService: AuthService) {}

  async ngOnInit() {
    const profile = await this.authService.getUserProfile();
    if (profile) {
      this.username = profile.username;
    }
  }
}

```

```

<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Bem-vindo(a)!</ion-title>
  </ion-toolbar>

```

```

</ion-header>

<ion-content class="ion-padding">
  <div class="welcome-card">
    @if (username) {
      <h1>Olá, {{ username }}!</h1>
    }
    <p>Que bom te ver por aqui. O que você gostaria de fazer hoje?</p>
  </div>
</ion-content>

```

▼ Error Refresh Token

sempre retorna um Observable (do RxJS).

```
this.http.post<any>(url, body)
```

```

// Renova access token usando refresh token
async refreshToken() {
  const refresh = await this.storage.get(this.refreshTokenKey);

  const refreshObservable = this.http.post<any>(`${this.apiUrl}/auth/refresh/`, { refresh });

  console.log('[AuthService] Refresh token atual:', refresh);
  console.log('[AuthService] refreshObservable:', refreshObservable);

  // lastValueFrom transforma o Observable em uma Promise
  return await lastValueFrom(refreshObservable);
}

```

- **Promise**

- Representa **uma única resposta**.
- OK ou NAO OK **uma vez só**.
- Sintaxe: `await fetch(...).then(...).catch(...)` .

- **Observable**

- Pode emitir **múltiplos valores ao longo do tempo**.
- Mais poderoso: você pode **cancelar, mapear, combinar** várias streams.
- Sintaxe: `observable$.pipe(map(...), catchError(...))` .

Angular **HttpClient** sempre retorna **Observables por padrão**, porque a biblioteca quer te dar mais controle sobre fluxos de dados.

Por que usar `lastValueFrom` ?

- `http.post` retorna um **Observable**.
- `async/await` , que só funciona com **Promises**.
- `lastValueFrom` pega o **último valor emitido pelo Observable** e transforma em Promise.

No interceptor

- Interceptor precisa lidar com **requisições que podem falhar e precisar de refresh**.
- Por isso `from(...)` :
 - Converte a Promise (do `refreshToken` ou `setTokens`) em Observable, para poder usar `switchMap` e continuar o fluxo.
- `switchMap` é usado para **substituir um Observable por outro**, garantindo que a requisição original espere o token novo antes de continuar.

Vamos deixar flexível a parte de Lifetime tempo de acesso do token

tempos de expiração, você precisa adicionar no `settings.py` algo assim:

```
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15), # tempo do a
    ccess token
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7), # tempo do ref
    resh token
    'ROTATE_REFRESH_TOKENS': True, # gera refresh token
    novo ao renovar
    'BLACKLIST_AFTER_ROTATION': True, # invalida refresh a
    ntigo
    'AUTH_HEADER_TYPES': ('Bearer',), # cabeçalho Authoriz
    ation: Bearer <token>
}
```

Layout

▼ Pagina Inicio, Minhas consultas, Agendar e Tabs

html

```
<ion-header>
  <ion-toolbar class="custom-toolbar">
    <ion-title>Bem-vindo(a)!</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">

  <!-- Boas-vindas ->
  <div class="welcome-card" style="text-align:center; margin-bottom:
  30px;">
```

```

    @if (username) {
      <h1 style="font-size: 2rem; margin-bottom: 10px;">Olá, {{ usernam
e }}!</h1>
    }
    <p>Que bom te ver por aqui. O que você gostaria de fazer hoje?</p>
  >
</div>

<!-- Grid centralizado →
<ion-grid class="simple-grid">
  <ion-row class="ion-justify-content-center" style="flex-direction: c
olumn; gap: 15px;">
    <ion-col size="auto" class="grid-item">Agendadas: 3</ion-col>
    <ion-col size="auto" class="grid-item">Concluídas: 5</ion-col>
    <ion-col size="auto" class="grid-item">Canceladas: 1</ion-col>
  </ion-row>
</ion-grid>

<!-- Espaço flexível →
<div style="flex: 1;"></div>

<!-- Banner de dica no bottom →
<ion-card class="banner-card" style="position: absolute; bottom: 20
px; left: 10px; right: 10px;">
  <ion-card-header>
    <ion-card-title>Dica do Dia</ion-card-title>
  </ion-card-header>
  <ion-card-content>
    Não se esqueça de vacinar seu pet no mês correto!
  </ion-card-content>
</ion-card>

</ion-content>

```

SCSS

```
.simple-grid {
  ion-row {
    display: flex;
    flex-direction: column;
    gap: 12px;

    .grid-item {
      background: #fff;
      color: #333;
      padding: 16px;
      text-align: center;
      border-radius: 10px;
      box-shadow: 0 2px 6px rgba(0, 0, 0, 0.08);
      font-weight: 600;
      width: 120px;
    }

    .grid-item:nth-child(1) { border-left: 4px solid #60b12a; }
    .grid-item:nth-child(2) { border-left: 4px solid #6c757d; }
    .grid-item:nth-child(3) { border-left: 4px solid #dc3545; }
  }
}

.banner-card {
  border-radius: 10px;
  background: #f9f9f9;
  position: absolute;
  bottom: 20px;
  left: 10px;
  right: 10px;

  ion-card-title {
    color: #60b12a;
    font-weight: 700;
  }
}
```

```
ion-card-content {  
  color: #555;  
  font-size: 14px;  
}  
}
```

global.scss

```
.custom-toolbar {  
  --background: #60b12a;  
  --color: #fff;  
  --border-radius: 4px;  
  --box-shadow: none;  
  font-weight: 600;  
  ion-icon {  
    font-size: 18px;  
  }  
}  
  
.custom-button {  
  --background: #60b12a;  
  --color: #fff;  
  --border-radius: 4px;  
  --box-shadow: none;  
  font-weight: 600;  
  ion-icon {  
    font-size: 18px;  
  }  
}
```

Pagina Minhas consultas

html

```

<ion-header>
  <ion-toolbar class="custom-toolbar">
    <ion-title>Minhas Consultas</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <ion-segment [(ngModel)]="filtroStatus" (ionChange)="filtrarConsultas()">
    <ion-segment-button value="todas"><ion-label>Todas</ion-label>
  </ion-segment-button>
    <ion-segment-button value="agendada"><ion-label>Agendadas</ion-label>
  </ion-segment-button>
    <ion-segment-button value="concluida"><ion-label>Concluídas</ion-label>
  </ion-segment-button>
  </ion-segment>

  <div *ngIf="consultasFiltradas.length === 0" class="sem-consultas">
    <ion-icon name="calendar-outline"></ion-icon>
    <h3>Nenhuma consulta encontrada</h3>
    <p>Você não possui consultas {{ filtroStatus !== 'todas' ? 'com este status' : '' }}</p>
  </div>

  <div class="consultas-grid">
    @for (consulta of consultasFiltradas; track $index) {
      <ion-card class="consulta-card" [ngClass]="status-' + (consulta.status || 'agendada').toLowerCase()">
        <div class="status-indicator">
          <ion-badge color="light" style="color:#60b12a">{{ getStatusText(consulta.status) }}</ion-badge>
        </div>
        <ion-card-header>
          <ion-card-subtitle>

```

```

        <ion-icon name="calendar-outline"></ion-icon>
        {{ consulta.data | date:'dd/MM/yyyy HH:mm' }}
    </ion-card-subtitle>
    <ion-card-title class="pet-nome">{{ consulta.animal.nome }}</ion-
n-card-title>
    </ion-card-header>
    <ion-card-content>
        <div class="info-row"><ion-icon name="person-outline"></ion-i
con>{{ consulta.animal.dono.nome }}</div>
        <div class="info-row"><ion-icon name="paw-outline"></ion-ico
n>{{ consulta.animal.tipo_especie }}</div>
        <div class="info-row"><ion-icon name="medkit-outline"></ion-i
con>Dr(a). {{ consulta.veterinario.nome }}</div>
    </ion-card-content>
    </ion-card>
}
</div>
</ion-content>

```

SCSS

```

.consultas-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 16px;
  margin-top: 16px;
}

.consulta-card {
  margin: 0;
  border-radius: 12px;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
  overflow: hidden;
  transition: transform .2s;
}

```

```

background: #fff;

&.hover { transform: translateY(-4px); }

.status-indicator {
  position: absolute;
  top: 8px; right: 8px;
}

ion-card-header {
  ion-card-subtitle {
    font-size: 14px;
    font-weight: 600;
    color: #60b12a;
    display: flex; align-items: center;

    ion-icon { margin-right: 6px; }
  }
  .pet-nome {
    font-size: 18px;
    font-weight: 700;
    margin-top: 2px;
  }
}

ion-card-content {
  .info-row {
    display: flex;
    align-items: center;
    margin-bottom: 6px;
    color: #555;

    ion-icon {
      margin-right: 8px;
      color: #60b12a;
    }
  }
}

```

```

    }
  }
}

.status-agendada { border-left: 4px solid #60b12a; }
.status-concluida { border-left: 4px solid #2ecc71; }
.status-cancelada { border-left: 4px solid #e74c3c; }

.sem-consultas {
  text-align: center;
  padding: 40px 0;
  color: #666;

  ion-icon { font-size: 48px; color: #bbb; margin-bottom: 12px; }
  h3 { font-size: 18px; margin: 0; color: #333; }
  p { margin-top: 6px; }
}

ion-segment {
  margin-bottom: 16px;
  border-radius: 8px;
  padding: 4px;

  ion-segment-button {
    min-height: 36px;
    --border-radius: 6px;
  }
}

```

ts

```

import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';

```

```

import { FormsModule } from '@angular/forms';
import {
  IonContent,
  IonHeader,
  IonTitle,
  IonToolbar,
  IonList,
  IonCard,
  IonCardHeader,
  IonCardSubtitle,
  IonCardTitle,
  IonCardContent,
  IonSegment,
  IonSegmentButton,
  IonLabel,
  IonBadge,
  IonIcon
} from '@ionic/angular/standalone';
import { ApiService, Consulta } from '../services/api';
import { LoadingController } from '@ionic/angular';
import { addIcons } from 'ionicons';
import { calendarOutline, personOutline, pawOutline, medkitOutline } from 'ionicons/icons';

@Component({
  selector: 'app-consultas',
  templateUrl: './consultas.page.html',
  styleUrls: ['./consultas.page.scss'],
  standalone: true,
  imports: [
    IonCardContent, IonCardTitle, IonCardSubtitle, IonCardHeader, IonCard,
    IonContent, IonHeader, IonTitle, IonToolbar, CommonModule,
    FormsModule, IonSegment, IonSegmentButton, IonLabel, IonBadge,
    IonIcon
  ]
})

```

```

    })
    export class ConsultasPage implements OnInit {

        consultas: Consulta[] = [];
        consultasFiltradas: Consulta[] = [];
        filtroStatus: string = 'todas';

        constructor(private api: ApiService, private loadingCtrl: LoadingController) {
            addIcons({
                calendarOutline,
                personOutline,
                pawOutline,
                medkitOutline
            });
        }

        ngOnInit() {
            this.listConsultas(); // Carrega lista de consultas
        }

        // Método para filtrar consultas por status
        filtrarConsultas() {
            if (this.filtroStatus === 'todas') {
                this.consultasFiltradas = [...this.consultas];
            } else {
                this.consultasFiltradas = this.consultas.filter(consulta =>
                    (consulta.status || 'agendada').toLowerCase() === this.filtroStatus
                );
            }
        }

        // Método para obter o texto do status
        // nao temos esse parametro ainda vindo do DB mas podemos deixar aqui pra usar
    }

```

```

getStatusText(status: string | undefined): string {
  switch ((status || 'agendada').toLowerCase()) {
    case 'agendada': return 'Agendada';
    case 'concluida': return 'Concluída';
    case 'cancelada': return 'Cancelada';
    default: return 'Pendente';
  }
}

async listConsultas() {

  // Cria e exibe o loading
  const loading = await this.loadingCtrl.create({
    message: 'Carregando consultas...',
    spinner: 'crescent',
    backdropDismiss: false
  });

  await loading.present(); // mostra

  this.api.listConsultas().subscribe({
    next: async (data) => {
      console.log("Lista de Consultas:", data);
      this.consultas = data;

      // Aplica o filtro inicial
      this.filtrarConsultas();

      await loading.dismiss()
    },
    error: async (error) => {
      console.error('Erro ao buscar consultas:', error);

      await loading.dismiss()
    }
  });
}

```

```
}  
}
```

Pagina Agendar Consulta

```
animalSelecionado: any = null;  
veterinarioSelecionado: any = null;  
  
constructor(private modalCtrl: ModalController) { }  
  
async abrirModalAnimais() {  
  // tira foco de qualquer botão/input ativo da tela  
  (document.activeElement as HTMLElement)?.blur();  
  
  const modal = await this.modalCtrl.create({  
    component: ListaAnimaisModal,  
    componentProps: {  
      animais: this.animais  
    }  
  });  
  
  modal.onDidDismiss().then((result) => {  
    if (result.data) {  
      console.log('Animal selecionado:', result.data);  
      this.animalSelecionado = result.data;  
    }  
  });  
  
  return await modal.present();  
}  
  
async abrirModalVeterinarios() {  
  // tira foco de qualquer botão/input ativo da tela  
  (document.activeElement as HTMLElement)?.blur();
```

```

const modal = await this.modalCtrl.create({
  component: ListaVeterinariosModal,
  componentProps: {
    veterinarios: this.veterinarios
  }
});
await modal.present();

modal.onDidDismiss().then((result) => {
  if (result.data) {
    console.log('Veterinario selecionado:', result.data);
    this.veterinarioSelecionado = result.data;
  }
});
}

```

```

.form-item {
  margin-bottom: 16px;
}

```

```

<form (ngSubmit)="agendar()">

  <!-- Seleção de Animal -->
  <ion-item button (click)="abrirModalAnimais()" class="form-item">
    <ion-avatar slot="start">
      <img [src]="animalSelecionado?.foto || 'https://placeholder.co/400'">
    </ion-avatar>
    <ion-label>
      {{ animalSelecionado?.nome || 'Selecione o animal' }}
    </ion-label>
  </ion-item>

```

```

<!-- Seleção de Veterinário →
<ion-item button (click)="abrirModalVeterinarios()" class="form-item">
  <ion-avatar slot="start">
    <img [src]="veterinarioSelecioneado?.foto || 'https://placeholder.co/400'">
  </ion-avatar>
  <ion-label>
    {{ veterinarioSelecioneado?.nome || 'Selecione o veterinário' }}
  </ion-label>
</ion-item>

```

ListaAnimaisModal

```

import { CommonModule } from '@angular/common';
import { Component, Input } from '@angular/core'; // ← Importe Input aqui
import { ModalController } from '@ionic/angular';
import { IonContent, IonToolbar, IonHeader, IonTitle, IonButtons, IonButton, IonList, IonItem, IonAvatar, IonLabel, IonNote, IonText } from '@ionic/angular/standalone';

@Component({
  selector: 'app-lista-animais',
  imports: [
    CommonModule,
    IonContent,
    IonToolbar,
    IonHeader,
    IonTitle,
    IonButtons,
    IonButton,

```

```

    IonList,
    IonItem,
    IonAvatar,
    IonLabel,
    IonNote,
    IonText,
    IonTitle,
    IonToolbar,
  ],
  standalone: true,
  template: `
    <ion-header>
      <ion-toolbar>
        <ion-title>Selecione o Animal</ion-title>
        <ion-buttons slot="end">
          <ion-button (click)="fechar()">Fechar</ion-button>
        </ion-buttons>
      </ion-toolbar>
    </ion-header>

    <ion-content>
      <ion-list>
        @for (animal of animais; track $index) {
          <ion-item button (click)="selecionar(animal)">
            <ion-avatar slot="start">
              <img src='https://placeholder.co/400'>
            </ion-avatar>
            <ion-label>
              <strong>{{animal.nome}}</strong><br />
              <ion-text>Especie: {{animal.tipo_especie}}</ion-text><br />
              <ion-note color="medium" class="ion-text-wrap">
                Raça: {{animal.raca}}
              </ion-note>
            </ion-label>
          </ion-item>
        }
      </ion-list>
    </ion-content>
  `

```

```

        </ion-list>
      </ion-content>
    ],
    providers: [ModalController]
  })
  export class ListaAnimaisModal {
    @Input() animais: any[] = [];

    constructor(private modalCtrl: ModalController) {}

    selecionar(animal: any) {
      this.modalCtrl.dismiss(animal);
    }

    fechar() {
      this.modalCtrl.dismiss();
    }
  }
}

```

ListaVeterinariosModal

```

import { Veterinario } from '../services/api';
import { CommonModule } from '@angular/common';
import { Component, Input } from '@angular/core';
import { ModalController } from '@ionic/angular';
import { IonHeader, IonToolbar, IonTitle, IonButtons, IonButton, IonContent, IonList, IonItem, IonAvatar, IonLabel, IonText, IonNote } from '@ionic/angular/standalone';

@Component({
  selector: 'app-lista-veterinarios',
  imports: [
    CommonModule,
    IonHeader,

```

```

    IonToolbar,
    IonTitle,
    IonButtons,
    IonButton,
    IonContent,
    IonList,
    IonItem,
    IonAvatar,
    IonLabel,
    IonText,
    IonNote
  ],
  standalone: true,
  template: `
    <ion-header>
      <ion-toolbar>
        <ion-title>Selecione o Veterinário</ion-title>
        <ion-buttons slot="end">
          <ion-button (click)="fechar()">Fechar</ion-button>
        </ion-buttons>
      </ion-toolbar>
    </ion-header>

    <ion-content>
      <ion-list>
        @for (vet of veterinarios; track $index) {
          <ion-item button (click)="selecionar(vet)">
            <ion-avatar slot="start">
              <img src='https://placeholder.co/400'>
            </ion-avatar>
            <ion-label>
              <strong>{{ vet.nome }}</strong><br />
              <ion-text>CRMV: {{vet.crmv}}</ion-text><br />
              <ion-note color="medium" class="ion-text-wrap">
                Especialidade: {{vet.especialidade}}
              </ion-note>
            </ion-label>
          </ion-item>
        }
      </ion-list>
    </ion-content>
  `

```

```

        </ion-label>
      </ion-item>
    }
  </ion-list>
</ion-content>
',
},
providers: [ModalController] // Adicionar ModalController aos provide
rs
})
export class ListaVeterinariosModal {
  @Input() veterinarios: Veterinario[] = [];

  constructor(private modalCtrl: ModalController) { }

  seleccionar(vet: Veterinario) {
    this.modalCtrl.dismiss(vet);
  }

  fechar() {
    this.modalCtrl.dismiss();
  }
}

```

Config Tabs

```

.custom-tab-bar {
  --background: #fff;
  --color: #aaa; // inativo
  --color-selected: #60b12a; // ativo

  border-top: 1px solid #eee;
  padding: 4px 0;

  ion-tab-button {

```

```

display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
font-size: 12px;

ion-icon {
  font-size: 26px;
  transition: color 0.2s, transform 0.2s;
}

&.tab-selected ion-icon {
  color: #60b12a;
  transform: scale(1.2);
}

ion-label {
  font-weight: 600;
  color: inherit;
}
}
}
}

```

▼ Configurar Usuário autenticado, Lista e Animais

Vamos adicionar relacionamento com User e Cliente.

- Cliente vai conseguir se autenticar no app

models.py

```

from django.contrib.auth.models import User

# Tabela de clientes
class Cliente(models.Model):
    usuario = models.OneToOneField(User, on_delete=models.CASCADE)

```

```
E, related_name='cliente', null=True, blank=True)
...
```

serializers.py

```
from django.contrib.auth.models import User

class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'first_name', 'last_name']
        read_only_fields = ['id']

class ClienteSerializer(serializers.ModelSerializer):
    usuario = UserSerializer(read_only=True)
```

views.py

```
def get_queryset(self):
    user = self.request.user
    if user.is_staff:
        return Consulta.objects.all()
    return Consulta.objects.filter(animal__dono__usuario=user)

def get_queryset(self):
    user = self.request.user
    if user.is_staff:
        return Animal.objects.all()
    return Animal.objects.filter(dono__usuario=user)
```

Vamos testar,

Script para criar usuarios

```
management >> commands >> create_users_for_clients.py
```

```
from django.core.management.base import BaseCommand
from django.contrib.auth.models import User
from core.models import Cliente

class Command(BaseCommand):
    help = 'Cria usuários para clientes que não possuem um usuário associado'

    def handle(self, *args, **options):

        # Obter todos os clientes sem usuário associado
        clientes_sem_usuario = Cliente.objects.filter(usuario__isnull=True)

        if not clientes_sem_usuario.exists():
            self.stdout.write(self.style.SUCCESS('Todos os clientes já possuem usuários associados.))
            return

        contador = 0

        for cliente in clientes_sem_usuario:

            if cliente.email:
                username = cliente.email.split('@')[0] if cliente.email else None
            else:
                username = cliente.nome

            senha = cliente.nome[:2] + '123123@' # Exemplo jo123123@

            # Criar usuário
            user = User.objects.create_user(
                username=username,
                email=cliente.email,
```

```

        password=senha,
        first_name=cliente.nome
    )

    # Associar usuário ao cliente
    cliente.usuario = user
    cliente.save()

    contador += 1
    self.stdout.write(self.style.SUCCESS(f'Usuário criado para {cliente.nome}'))

    self.stdout.write(self.style.SUCCESS(f'Total de {contador} usuários criados com sucesso!'))

```

forms

```

def save(self, commit=True):
    """
    Cria usuários para cliente que não possuem um usuário associado
    """
    cliente = super().save(commit=False)
    print("[FORMS linha 32]", cliente)
    if not cliente.usuario: # usuario == null, vazio
        if cliente.email:
            username = cliente.email.split('@')[0]
        else:
            username = cliente.nome

        senha = cliente.cpf
        user = User.objects.create_user(
            username=username,
            email=cliente.email,

```

```

        password=senha,
        first_name=cliente.nome
    )
    cliente.usuario = user

    if commit:
        cliente.save()
    return cliente

```

Correção no data Timezone +3

```

class ConsultaForm(forms.ModelForm):
    ...
    widgets = {
        'data': forms.DateTimeInput(
            attrs={'type': 'datetime-local', 'class': 'form-control'},
            format='%Y-%m-%dT%H:%M'
        ),
        ...
    }

```

```

eventClick: function(info) {
    if (info.event.title === "Disponível") {
        // Selecionar horário disponível
        //const dataFormatada = info.event.start.toISOString().slice(0, 16).replace('T', ' ');

        const data = info.event.start

        console.log("data click", data)

        // ajustar formato para datetime-local
        const ano = data.getFullYear();
        const mes = String(data.getMonth() + 1).padStart(2, '0');
    }
}

```

```

const dia = String(data.getDate()).padStart(2, '0');
const horas = String(data.getHours()).padStart(2, '0');
const minutos = String(data.getMinutes()).padStart(2, '0');

const dataFormatada = `${ano}-${mes}-${dia}T${horas}:${minutos}s`;

console.log("[DATA SELECIONADA]", dataFormatada)

document.querySelector('[name="data"]').value = dataFormatada;
$('#dataDoctorModal').modal('hide');
} else {
  alert("Consulta já agendada: " + info.event.title);
}
}

```

▼ Lista de Agendar consulta, status e configuração

Agendar consulta Status

```

class Consulta(models.Model):
    class StatusConsulta(models.TextChoices):
        AGENDADA = 'Agendada'
        CONCLUIDA = 'Concluida'
        CANCELADA = 'Cancelada'

    ...

    status = models.CharField(max_length=15, choices=StatusConsulta.
choices, default=StatusConsulta.AGENDADA)

    created_at = models.DateTimeField(auto_now_add=True) # Data de

```

criação

```
updated_at = models.DateTimeField(auto_now=True) # Data de atualização
```

Status Badged

```
.status-indicator {  
  position: absolute;  
  top: 8px; right: 8px;  
  
  ion-badge.badge-agendada {  
    --color: #fff;  
    --background: #ddd60d;  
  }  
  ion-badge.badge-concluida {  
    --color: #fff;  
    --background: #60b12a;  
  }  
  ion-badge.badge-cancelada {  
    --color: #fff;  
    --background: #e74c3c;  
  }  
}
```

Pagina Inicio puxar informações.

`action` é método extra no seu `ViewSet` além dos métodos padrão (`list`, `retrieve`, `create`, etc). Ele te permite criar endpoints customizados.

```
@action(detail=False)  
def resumo_consultas(self, request):
```

```

qs = self.get_queryset()

resumo = {
    "todas": qs.count(), # 4
    "agendadas": qs.filter(status='Agendada').count(), # 5
    "concluidas": qs.filter(status='Concluida').count(),
    "canceladas": qs.filter(status='Cancelada').count(),
}

return Response(resumo)

```

▼ Agendar Consulta, fullCalendar

<https://fullcalendar.io/docs/angular>

▼ Implementa Fullcalendar

```

npm install --save \
  @fullcalendar/core \
  @fullcalendar/angular \
  @fullcalendar/daygrid \
  @fullcalendar/interaction

```

Gera um componente

```
ng generate component components/calendar
```

calendar.component.ts

```

import { Component, EventEmitter, Input, Output } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FullCalendarModule } from '@fullcalendar/angular';
import { CalendarOptions } from '@fullcalendar/core';
import dayGridPlugin from '@fullcalendar/daygrid';

```

```

import timeGridWeekPlugin from '@fullcalendar/timegrid';
import interactionPlugin from '@fullcalendar/interaction';
import ptBrLocale from '@fullcalendar/core/locales/pt-br';

@Component({
  selector: 'app-calendar',
  standalone: true,
  imports: [CommonModule, FullCalendarModule],
  templateUrl: './calendar.component.html',
  styleUrls: ['./calendar.component.scss'],
})
export class CalendarComponent {
  @Input() events: any[] = [];
  @Output() dateClicked = new EventEmitter<string>();
  @Output() eventClicked = new EventEmitter<any>();

  calendarOptions: CalendarOptions = {
    plugins: [
      dayGridPlugin,
      timeGridWeekPlugin,
      interactionPlugin],
    initialView: 'timeGridWeek',

    locale: ptBrLocale,
    selectable: true,
    events: [],

    weekends: false, // não mostra sábado e domingo

    // Configurações de horário
    slotMinTime: '08:00:00',
    slotMaxTime: '17:00:00',
    slotDuration: '01:00:00',
    allDaySlot: false,

    // Horário de trabalho (excluindo almoço)

```

```

    businessHours: [
      { daysOfWeek: [1, 2, 3, 4, 5], startTime: '08:00', endTime: '12:00' },
      { daysOfWeek: [1, 2, 3, 4, 5], startTime: '13:00', endTime: '17:00' }
    ],

    // Bloquear datas passadas
    // validRange: {
    //   start: new Date().toISOString().split('T')[0] // Data atual em formato YYYY-MM-DD
    // },

    dateClick: (arg) => this.dateClicked.emit(arg.dateStr),
    eventClick: (arg) => this.eventClicked.emit(arg.event)
  };

  ngOnChanges() {
    if (this.events) {
      this.calendarOptions = {
        ...this.calendarOptions,
        events: this.events // atualiza quando `events` mudar
      };
    }
  }
}

```

html

```
<full-calendar [options]="calendarOptions"></full-calendar>
```

por exemplo no agendar consulta ficaria assim:

```
<app-calendar
  [events]="meusEventos"
```

```

      (dateClicked)="onDateClick($event)"
      (eventClicked)="onEventClick($event)">
    </app-calendar>

    <ion-item>
      <ion-label>Data e Hora Selecionada:</ion-label>
      <ion-input [value]="dataSelecionada" [disabled]="true" required
    ></ion-input>
    </ion-item>

```

ts

```

meusEventos = [
  {
    id: 1,
    title: "Disponível",
    start: '2025-09-15T14:00:00',
    color: "#28a745",
  },
  {
    id: 2,
    title: "Disponível",
    start: '2025-09-15T15:00:00',
    color: "#28a745",
  },
  {
    id: 3,
    title: "Disponível",
    start: '2025-09-16T10:00:00',
    color: "#28a745",
  },
  {
    id: 4,
    title: "Disponível",
    start: '2025-09-17T09:00:00',
    color: "#28a745",
  },
]

```

```

    }
  ];

  // data
  dataSelecionada: string = '';

  onEventClick(event: any) {
    console.log('Evento clicado ', event.start);

    // Atualiza a data na consulta para ser enviada ao servidor
    this.consulta.data = event.start;

    // Formata a data para exibição amigável
    const data = new Date(event.start);
    const dia = data.getDate().toString().padStart(2, '0');
    const mes = (data.getMonth() + 1).toString().padStart(2, '0');
    const ano = data.getFullYear();
    const hora = data.getHours().toString().padStart(2, '0');
    const minutos = data.getMinutes().toString().padStart(2, '0');
    const periodo = data.getHours() < 12 ? 'da manhã' : data.getHours() < 18 ? 'da tarde' : 'da noite';

    // Cria a string formatada e armazena na propriedade
    this.dataSelecionada = `${dia}/${mes}/${ano} às ${hora}:${minutos} ${periodo}`;

    console.log('Data formatada:', this.dataSelecionada);
  }

```

scss

```

@use 'sass:color';

::ng-deep {
  // Título do calendário
  .fc .fc-toolbar-title {

```

```

    font-size: 18px;
    color: #333; // cor do texto
    font-weight: bold;
}

// Cabeçalhos dos dias da semana
.fc .fc-col-header-cell-cushion {
    color: #555;
    font-weight: 600;
}

// Células do calendário
.fc .fc-daygrid-day {
    background-color: #f9f9f9;
    border: 1px solid #ddd;
}

// Eventos
.fc .fc-event {
    background-color: #3788d8;
    border-color: #3788d8;
    color: white;
    border-radius: 4px;
}

// Hoje
.fc .fc-day-today {
    background-color: rgba(255, 220, 40, 0.15) !important;
}

// Botões da barra de ferramentas
.fc .fc-button-primary {
    background-color: #3788d8;
    border-color: #3788d8;

    &:hover {

```

```

        background-color: color.adjust(#3788d8, $lightness: -10%);
        border-color: color.adjust(#3788d8, $lightness: -10%);
    }
}

// Linhas de hora
.fc .fc-timegrid-slot {
    height: 40px;
    border-bottom: 1px solid #eee;
}

// Horário de trabalho
.fc .fc-timegrid-col.fc-day-past {
    background-color: rgba(0, 0, 0, 0.04);
}

// Horário de almoço (você pode adicionar uma classe específica
no componente)
.fc .lunch-time {
    background-color: rgba(255, 235, 230, 0.5);
}

@media screen and (max-width: 768px) {
    .fc .fc-toolbar {
        flex-direction: column;
    }

    .fc .fc-toolbar-title {
        font-size: 16px;
    }
}
}

```

▼ Horário disponíveis

Vamos mostrar as consultas disponíveis dos Médicos

primeiro passo é implementar um action para puxar as consultas dos Medicos veterinarios

voce vão perceber que é mesma que utilizamos no frontend do django. rsrs vai essa mesmo.

```
@action(detail=False, methods=['get'])
def eventos_veterinario(self, request):
    from django.utils.timezone import localtime, make_aware, get_current_timezone
    from datetime import datetime, time, timedelta # time é mais li mpo que datetime.min.time()

    veterinario_id = request.query_params.get("veterinario")
    if not veterinario_id:
        return Response([], status=200)

    # Consultas agendadas já no formato necessário
    consultas = (
        Consulta.objects.filter(veterinario_id=veterinario_id)
        .select_related("animal", "veterinario")
    )

    consultas_dict = {
        f"{localtime(c.data).strftime('%Y-%m-%d')}-{localtime(c.data).hour}": c.id for c in consultas
    }

    print(consultas_dict)

    eventos = [
        {
            "id": c.id,
            "title": f"{c.animal.nome} - {c.veterinario.nome}",
            "start": localtime(c.data).strftime("%Y-%m-%dT%H:%M:%S"), # 2: Converta para o fuso local
```

```

        "color": "#dc3545", # vermelho
    }
    for c in consultas
]

hoje = localtime().date() # Use localtime() para pegar a data a
tual no fuso correto
horarios = [8, 9, 10, 11, 13, 14, 15, 16]

for i in range(7):
    data = hoje + timedelta(days=i)
    if data.weekday() >= 5: # Pula sábado e domingo
        continue

    for hora in horarios:
        key = f"{data.strftime('%Y-%m-%d')}-{hora}"
        print(key)
        if key not in consultas_dict:
            slot_inicio = make_aware(datetime.combine(data, time
(hour=hora)), get_current_timezone()) # timezone são paulo, portu
gal

            eventos.append(
                {
                    "id": f"disp-{data}-{hora}",
                    "title": "Disponível",
                    "start": slot_inicio.strftime("%Y-%m-%dT%H:%
M:%S"), # Formate o slot fuso horário
                    "color": "#28a745", # verde
                }
            )
    return Response(eventos)

```

core>> models

```

def __str__(self):
    veterinario = self.veterinario.nome if self.veterinario else 'desconhecido'

    # Converte a data/hora (que está em UTC) para o fuso horário local
    # definido no seu settings.py (America/Sao_Paulo)
    data_local = localtime(self.data)
    data_formatada = data_local.strftime('%d/%m/%Y às %H:%M')

    return f"Consulta de {self.animal.nome} com {veterinario} em {data_formatada}"

```

api.ts

```

getEventosVeterinario(veterinarioid: number): Observable<any[]>
{
    return this.http.get<any[]>(`${this.baseUrl}/consultas/eventos_veterinario/?veterinario=${veterinarioid}`);
}

```

agendar-consulta

```

meusEventos: any[] = []

async carregarEventosVeterinario(veterinarioid: number) {

    console.log('Carregando eventos do veterinário:', veterinarioid);

    // loading
    const loading = await this.loadingCtrl.create({
        message: 'Carregando...',
    });
}

```

```

    spinner: 'crescent',
    backdropDismiss: false
  });

  await loading.present();

  if (!veterinarioid) return;

  this.api.getEventosVeterinario(veterinarioid).subscribe({
    next: (eventos: any[]) => {
      this.meusEventos = eventos;
      loading.dismiss();
    },
    error: (error) => {
      console.error('Erro ao carregar eventos:', error);
      loading.dismiss();
    }
  });
}

```

Pronto, consultas disponiveis aparecendo.

Vamos da uma melhorada

```

private alertController: AlertController

if (event.title !== "Disponível") {
  this.alertController.create({
    header: 'Horário Indisponível',
    message: 'Este horário já está reservado para outra consult
a.',
    buttons: ['OK']
  }).then((alert: HTMLIonAlertElement) => alert.present());
  return;
}

```

calendar.component.ts

para mostrar a data que clicamos

```
eventoSelecionadold: string | number | null = null;

eventClick: (arg) => {
  // Salva o evento clicado
  this.eventoSelecionadold = arg.event.id;
  console.log('Evento clicado ', arg.event);

  // Atualiza a aparência do evento selecionado
  const calendarApi = arg.view.calendar;

  // Recarrega os eventos para aplicar o estilo
  calendarApi.getEvents().forEach(evt => {
    // Aplica estilo ao evento selecionado
    if (evt.id === this.eventoSelecionadold) {
      evt.setProp('backgroundColor', '#007bff');
      evt.setProp('borderColor', '#0056b3');
      evt.setProp('textColor', 'ffffff');
    }
    // Restaura o estilo dos eventos disponíveis não selecionados
    else if (evt.title === 'Disponível') {
      evt.setProp('backgroundColor', '#28a745');
      evt.setProp('borderColor', '#28a745');
      evt.setProp('textColor', 'ffffff');
    }
  });

  // Emite o evento para o componente pai
  this.eventClicked.emit(arg.event);
}
```

limpa formulário

```
this.consulta = {  
  animal: {} as Animal,  
  veterinario: {} as Veterinario,  
  data: new Date().toISOString(),  
  motivo: '',  
  observacoes: '',  
  status: 'Agendada'  
};  
this.animalSelecioneado = null;  
this.veterinarioSelecioneado = null;  
this.dataSelecioneada = '';  
this.meusEventos = [];
```

erro calendario nao limpa

calendar.component.ts

decorator do Angular que serve para pegar uma **referência direta** a um elemento do template ou a outro componente filho

```
@ViewChild('fullCalendar') calendarInstance!: FullCalendar;  
  
public clearEvents() {  
  if (this.calendarInstance) {  
    console.log("Calendar Instance !!!")  
    const api = this.calendarInstance.getApi();  
  
    api.removeAllEvents();  
    api.destroy();  
  
    console.log("Remove todos os events");  
  }  
}
```

```

console.log("Destroi e recria novamente");

setTimeout(() => {
  api.render();
  api.updateSize(); // 🖱️ força recalcular altura/largura
  window.dispatchEvent(new Event('resize')); // 🖱️ extra fallback
  console.log('Calendário limpo, recarregado e redimensionado.');
}, 100);
}
}

```

html

```

<full-calendar #fullCalendar [options]="calendarOptions"></full-calendar>

```

agendar.consulta vou chamar ele

```

@ViewChild(CalendarComponent) calendar!: CalendarComponent;

```

criar um função para limpa

```

resetFormCalendar() {
  // Limpa os dados do formulário
  this.consulta = {
    animal: {} as Animal,
    veterinario: {} as Veterinario,
    data: new Date().toISOString(),
    motivo: '',
    observacoes: '',
    status: 'Agendada'
  };
}

```

```

this.animalSelecionado = null;
this.veterinarioSelecionado = null;
this.dataSelecionada = '';
this.meusEventos = [];

this.calendar.clearEvents();
}

```

agendar

```

async agendar() {

    // no submit a hora
    // validação simples do horário: 08:00-12:00 ou 13:00-17:00
    const date = new Date(this.consulta.data);
    const hour = date.getHours();
    if (!((hour >= 8 && hour < 12) || (hour >= 13 && hour < 17))) {
        alert('Horário deve ser entre 08:00-12:00 ou 13:00-17:00');
        return;
    }

    console.log("Consulta a agendar:", this.consulta);

    // loading
    const loading = await this.loadingCtrl.create({
        message: 'Carregando...',
        spinner: 'crescent',
        backdropDismiss: false
    });

    await loading.present();

    this.api.agendarConsulta(this.consulta).subscribe({
        next: (data) => {
            console.log("Status:", data);
        }
    });
}

```

```

this.resetFormCalendar();

// this.router.navigate(['/tabs/agendar']);

loading.dismiss();

this.alertController.create({
  header: 'Sucesso',
  message: 'Consulta cadastrada com sucesso!',
  buttons: [{
    text: 'OK',
    handler: () => {
      this.router.navigate(['/tabs/consultas']);
    }
  }]
}).then(alert => alert.present());

},
error: (error) => {
  console.log("Status:", error);
  loading.dismiss();
  alert('Erro ao agendar!')
}
});

}

```

▼ Lista, Adicionar/Atualizar Pet

